THM TECHNISCHE HOCHSCHULE MITTELHESSEN
CAMPUS FRIEDBERG
IEM Informationstechnik-Elektrotechnik-Mechatronik

Bachelorarbeit

Portierung und Erweiterung eines Plasma-Systems zur einfachen Nutzung weiterer Hardware-Komponenten

zur Erlangung des akademischen Grades

Bachelor of Engineering (B.Eng.)

vorgelegt dem
Fachbereich Informationstechnik, Elektrotechnik und Mechatronik
der Technischen Hochschule Mittelhessen
von

Sergej Zuyev
geb. in Lugansk

17. September 2018

Referent der Arbeit:	Prof. Dr. Hartmut Weber
Korreferent der Arbeit:	Prof. Dr.-Ing. Martin Gräfe

In dieser Bachelorarbeit wird ein Plasma (MIPS-I) System um einem standardisierten System- und Peripheriebus erweitert und von einem FPGA des Herstellers Xilinx auf ein FPGA des Herstellers Altera/Intel portiert. Das Plasma (MIPS-I) System wird weiter ausgebaut, sodass eine Kommunikation mit neuen Hardware-Komponenten und IP-Cores von Altera/Intel möglich wird. Eine ARM-CPU auf dem Altera/Intel-FPGA und ein Linux-Betriebssystem werden konfiguriert. Für das Plasma-System werden eine Firmware und eine Beispielanwendung entworfen.

Mit den in dieser Bachelorarbeit entwickelten Linux-Anwendungen und einem Treiber wird die Korrektheit der Portierung und Funktionsfähigkeit der Schnittstellen zwischen den ARM- und Plasma (MIPS-I) Systemen gezeigt.

Selbstständigkeitserklärung

Ich erkläre, dass ich die eingereichte Bachelorarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die von mir angegebenen Quellen und Hilfsmittel nicht benutzt und die den benutzten Werken wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht habe.

Friedberg, den 17. September 2018

Sergej Zuyev

Inhaltsverzeichnis

Glossar **IV**

Abkürzungsverzeichnis **V**

Abbildungsverzeichnis **VII**

Tabellenverzeichnis **IX**

Quellcodeverzeichnis **X**

1. Einleitung **1**
1.1. Motivation 1
1.2. Problemstellung 1
1.3. Anforderungen 2
1.4. Organisation dieser Arbeit 2

2. Materialien **3**
2.1. ZedBoard development kit 3
2.2. DE1-SoC development kit 3
2.3. Quartus Prime 18.0 8
2.4. SoC FPGA Embedded Development Suite (SoC EDS) 8

3. Grundlagen **11**
3.1. System-On-Chip 11
3.2. Bussysteme 12
3.2.1. Topologien 12
3.2.2. Wishbone 14
3.2.3. Avalon 14
3.3. Plasma CPU 17
3.4. Device Tree 17
3.5. Lebenszyklus des Cyclone V SoCFPGA 18

4. Systemarchitektur **21**
4.1. Plasma 21
4.2. Plasma-SoC auf dem ZedBoard 21

4.3. Plasma auf dem DE1-SoC-Board 23

5. Umsetzung **25**
5.1. Entwicklung des Wishbone-basierten Systems 25
5.2. Entwicklung der Plasma-Firmware 30
5.2.1. Treiber 31
5.2.2. Dynamisches Speichermanagement 34
5.2.3. I/O 34
5.3. Portierung der Hardware für DE1-SoC 35
5.3.1. RAM 36
5.3.2. Register-File 39
5.3.3. Laufzeitverhalten 40
5.4. Erweiterung der Hardware für DE1-SoC 40
5.4.1. Avalon-Wishbone Bridge 40
5.4.2. Arbiter 40
5.4.3. Wishbone-Bus 43
5.4.4. Externer SDRAM 43
5.4.5. Parallel I/O 47
5.4.6. HPS 47
5.4.7. System ID 48
5.4.8. Adresszuordnung 48
5.4.9. Zusammenfassung 48
5.5. Unterstützende Software 52
5.6. Portierung der Plasma-Firmware und der Beispielanwendung 54
5.7. Einrichtung eines Linux-Betriebssystems 54
5.7.1. Devicetree 54
5.7.2. Bootloader 58
5.7.3. Kernel 59
5.7.4. Systemabbild 60
5.8. Entwicklung von Linux-Anwendungen für das HPS 60
5.9. Entwicklung eines Linux-Treibers für das HPS 61

6. Ergebnisse und Ausblick **63**
6.1. Zusammenfassung 63
6.2. Ausblick 64

A. Übersicht über elektronische Anhänge **68**

Glossar

Bridge

Eine Komponente, die eine Kommunikation zwischen zwei oder mehr Bussystemen untereinander ermöglicht.

Firmware

In elektronischen Geräten eingebettete Software.

GSM

Global System for Mobile Communications, früher Groupe Spécial Mobile: Mobilfunkstandard für volldigitale Mobilfunknetze.

Inferring

Fähigkeit des HDL-Compilers, aus Sprachkonstrukten Strukturelemente wie z.B RAM oder FIFO zu generieren.

Interconnect

Kommunikationsschnittstelle zwischen mehreren Geräten.

Abkürzungsverzeichnis

ARM Advanced RISC Machine.

AXI Advanced eXtensible Interface.

BSP Board Support Package.

CPU Central Processing Unit.

CSR Control/Status Register.

DDR SDRAM Double Data Rate Synchronous Dynamic Random Access Memory.

DMA Direct Memory Access.

DSP Digital Signal Processor.

ELF Executable and Linkable Format.

FPGA Field Programmable Gate Array.

GPIO General-Purpose Input/Output.

GPU Graphics Processing Unit.

HDL Hardware Definition Language.

HPS Hard Processor System.

IO Input/Output.

IoT Internet of Things.

IRQ Interrupt Request.

LPM library of parameterized modules.

LSB least significant bit.

MBR Master Boot Record.

MIF Memory Initialization File.

MIO Multiplexed I/O.

MIPS Microprocessor without Interlocked Pipeline Stages.

PHY **Phy**sical Layer.

PIO Parallel I/O.

PL Programmable Logic.

PLL phase locked loop.

PS Processing System.

RAM Random Access Memory.

RISC Reduced Instruction Set Computer.

RMW read-modify-write.

SoC System-on-Chip.

SOPC System-on-a-programmable-chip.

SPI Serial Peripheral Interface bus.

SRAM Static Random Access Memory.

UART Universal Asynchronous Receiver-Transmitter.

Abbildungsverzeichnis

2.1. ZedBoard Blockdiagramm [3, S. 3] 4
2.2. DE1-SoC Blockdiagramm [5, S. 9] 5
2.3. DE1-SoC Board-Layout [5, S. 7] 7
2.4. FPGA Configuration Mode Switch [5, S. 12] 7
2.5. Quartus Anwendungen 9

3.1. Crossbar (Switch) [8, S. 17] 13
3.2. Data flow [8, S. 17] 13
3.3. Point to point [8, S. 21] 13
3.4. Shared bus [8, S. 21] 14
3.5. Standard connection for timing diagrams [8, S. 14] 15
3.6. Avalon-Bus basiertes System [7, S. 7–6] 16
3.7. Plasma-CPU Blockdiagramm [1] 18
3.8. Bootvorgang [12, S. 95] 19

4.1. Blockdiagramm des Plasma im Ausgangszustand 22
4.2. Blockdiagramm des Plasma-SoC auf dem ZedBoard 22
4.3. Blockdiagramm des Plasma-SoC auf dem DE1-SoC 23
4.4. Blockdiagramm des Plasma-SoC auf dem DE1-SoC mit Verbindung zu weiteren IP-Cores und ARM Prozessor über den Avalon-Bus 24

5.1. DE1-SoC SystemBuilder 35
5.2. Quartus Boardeinstellungen 36
5.3. Plasma-System-on-Chip (SoC) IP-Core 37
5.4. Ausgelesener Speicherinhalt 38
5.5. Vergleich der RTL-Netzlisten 39
5.6. Wishbone↔Avalon-Bridge Timings 41
5.7. Parametrisierung des SDRAM-Controllers 45
5.8. Parametrisierung des *Clocks Signals for DE-Series Board Peripherals*-IP-Core 46
5.9. Mit SignalTap aufgezeichneter Zugriff auf den externen SDRAM . . . 47
5.10. Hard Processor System (HPS): Konfiguration 49
5.11. HPS SDRAM-Konfiguration 50
5.12. Tcl Scripts: hps_sdram_p0_pin_assignments.tcl 52

5.13. Das Gesamtsystem im Platform Designer 53
5.14. BSP-Editor 59
5.15. Zugriff auf das Reload-Register aus dem Terminal 62

Tabellenverzeichnis

2.1. FPGA Konfigurationsmodi [5, S. 13] 7

3.1. Avalon-MM Signale [9, S. 14–16] 17

5.1. Adresszuordnung 26
5.2. Allgemeine Register-Map 27
5.3. Interrupt Controller Register-Map 28
5.4. Interrupt Requests (IRQs) 28
5.5. Zähler Register-Map 28
5.6. Timer Register-Map 29
5.7. Timer Zeiteinheiten Kodierung und Reloadwert [ticks] 29
5.8. UART-Controller Register-Map 29
5.9. GPIO-Controller Register-Map 30
5.10. SPI Controller Register-Map 30
5.11. Display-Controller Register-Map 30
5.12. Control Register Bits 31
5.13. Allgemeine Treiberfunktionen 32
5.14. Treiberfunktionen des Display-Controllers 33
5.15. Treiberfunktionen des Timers 33
5.16. Treiberfunktionen des Zählers 33
5.17. Treiberfunktionen des Display-Controllers 34
5.18. Treiberfunktionen des Interrupt-Controllers 34
5.19. Avalon-MM und Wishbone Schnittstellen 42
5.20. Timing Parameter [22, S. 1] 44
5.21. Adresstabelle [22, S. 1] 44
5.22. Plasma-SoC Adresszuordnung 51

Quellcodeverzeichnis

5.1. Devicetree: Arbeitsspeicherdefinition 55
5.2. Devicetree: Adresstranslation in der HPS-Bridge 56
5.3. Devicetree: SysID-Definition 56
5.4. Devicetree: Komponenten im Plasma-SoC 57
5.5. Devicetree: Komponenten am Avalon-Bus 58

1. Einleitung

Diese Bachelorarbeit befasst sich mit der Portierung und Erweiterung eines „Plasma“-Systems [1] von Field Programmable Gate Arrays (FPGAs) des Herstellers Xilinx auf FPGAs des Herstellers Altera/Intel. Betrachtet werden hierbei sowohl die Hard- als auch Softwareaspekte.

Dieses Kapitel dient der Einführung in die Bachelorarbeit und die Problemstellung.

1.1. Motivation

Langfristig gesehen soll es möglich sein, einen hardware-basierten Beschleuniger zur Bildklassifizierung mittels binarisierter neuronaler Netze[2] in das zugrundeliegende „Plasma“-System zu integrieren. Bevor dies geschehen kann, muss das „Plasma“-System auf eine mit der Altera-OpenCL-Toolchain kompatible Hardware-Basis portiert und erweitert werden.

1.2. Problemstellung

Derzeit ist das Plasma-SoC nur auf dem Digilent Zedboard[1] mit einem Zynq-7000 FPGA lauffähig. Ziel der Arbeit ist es, eine Portierung für das Terasic DE1-SoC[2] Board mit einem Cyclone V FPGA vorzunehmen, und es soweit zu erweitern, dass eine einfache Kommunikation mit den Komponenten und Peripherie wie z.B Direct Memory Access (DMA)-Controller und externem Random Access Memory (RAM) möglich wird. Dafür soll ebenso die Software-Basis in Form von Firmware, Bertriebssystem und Hilfs-Software angepasst und erweitert werden.

[1] `http://zedboard.org/product/zedboard`
[2] `https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&No=836`

1.3. Anforderungen

Das Plasma-System soll auf dem FPGA des DE1-SoC-Board lauffähig sein und mit einem Advanced RISC Machine (ARM) Prozessor kommunizieren können. Außerdem soll ein auf dem DE1-SoC-Board auszuführendes Linux-Betriebssystem eingerichtet werden, welches auf das Plasma-System zugreifen können soll. Um die Funktionsfähigkeit der Lösung zu beweisen, sollen Beispielanwendungen entwickelt und auf dem Plasma-System und der ARM-CPU unter Linux ausgeführt werden. Zusätzlich soll ein für das Linux-Betriebssystem zu entwickelnder Treiber das Plasma-System ansteuern können.

1.4. Organisation dieser Arbeit

In Kapitel 2 werden die in dieser Bachelorarbeit verwendeten Materialien beschrieben.

Anschließend werden in Kapitel 3 die für ein besseres Verständnis notwendigen Grundlagen erklärt.

Darauffolgend werden in Kapitel 4 die Systemarchitekturen vorgestellt, die im Verlauf dieser Bachelorarbeit umgesetzt werden.

In Kapitel 5 wird die Umsetzung in Hardware und Software sowie die Erstellung eines lauffähigen Linux-Systems im Detail behandelt.

Die in vorangehenden Kapiteln gewonnenen Erkenntnisse werden in Kapitel 6 zusammengefasst und und ein Ausblick auf zukünftige Entwicklung vorgestellt.

2. Materialien

Dieses Kapitel befasst sich mit den in dieser Bacherlorarbeit verwendeten Materialien. Dazu gehört die Hardware und die Software.

2.1. ZedBoard development kit

Das ZedBoard von Digilent ist ein Entwicklungs- und Evaluationsboard mit einem verbauten Xilinx Zynq-7000 SoC mit Peripherie. Das Zynq SoC ist in 3 Blöcke aufgeteilt, wie in Abb. 2.1 detailliert aufgeführt ist:

- Multiplexed I/O (MIO)
- Processing System (PS)
- Programmable Logic (PL)

Das PS verfügt über einen ARM-Cortex™A9 Prozessor und kann auf den Double Data Rate Synchronous Dynamic Random Access Memory (DDR SDRAM) zugreifen. Das PL stellt ein FPGA mit Peripherie dar und kann sowohl als domänenspezifischer Beschleuniger für das PS als auch als eine autarke Einheit konfiguriert werden. Das MIO fasst die sowohl von der PS als auch PL nutzbare Peripherie - exklusiver Zugriff vorausgesetzt - zusammen.

2.2. DE1-SoC development kit

Das DE1-SoC Board von Terasic ist ein Eintwicklungs- und Evaluationsboard mit einem verbauten Cyclone V FPGA-SoC.

Wenn man das in Abb. 2.2 abgebildete Blockdiagramm betrachtet und mit dem Blockdiagramm des ZedBoard in Abb. 2.1 vergleicht, so fallen starke Ähnlichkeiten auf. Diese sind darin begründet, dass bei beiden FPGAs ein ähnliches Konzebt verfolgt wird, bei dem sowohl ein ARM-Prozessor als auch ein FPGA mit Peripherie

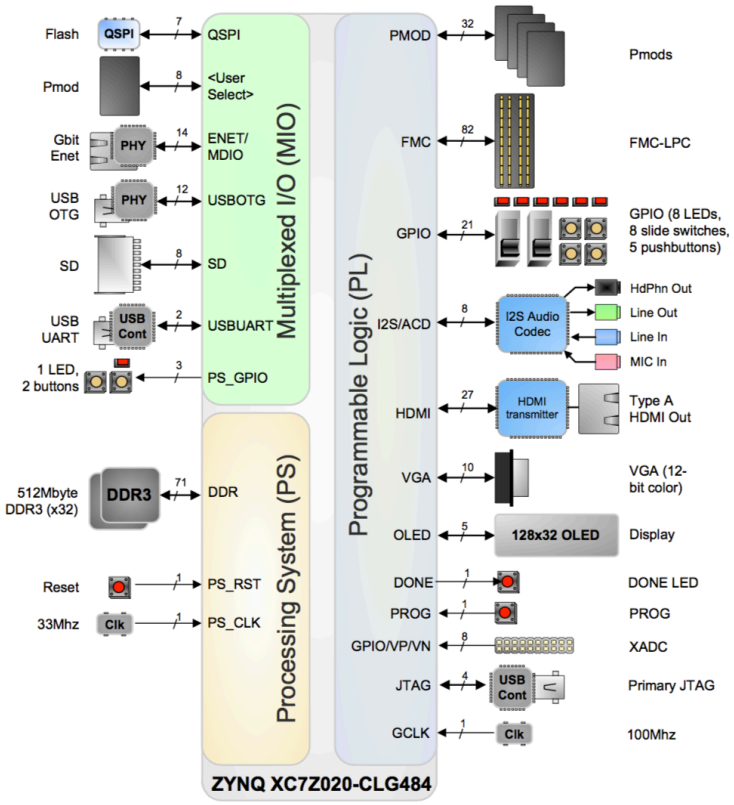


Abbildung 2.1.: ZedBoard Blockdiagramm [3, S. 3]

zur Verfügung gestellt wird. Diese Architektur wird in der Altera-Terminologie „SoC FPGA“ bzw. „SoCFPGA“ genannt [4, S. 1].

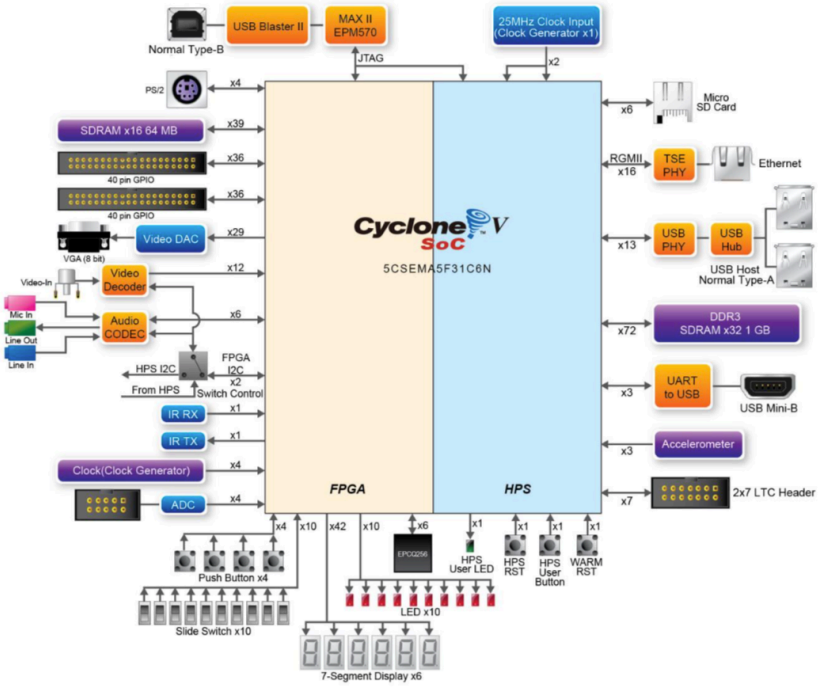


Abbildung 2.2.: DE1-SoC Blockdiagramm [5, S. 9]

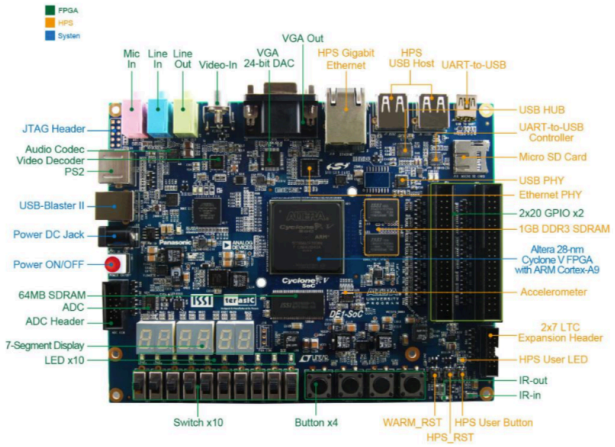
Auf dem DE1-SoC-Board stehen folgende Komponenten zur Verfügung [5]:

- FPGA
  - Altera Cyclone®V SE FPGA
  - 64 MB SDRAM (16 bit Datenbus)
  - 4 Taster
  - 10 Schalter
  - 10 rote LEDs
  - 6 7-Segment-Anzeigen
  - 4 50 MHz Oszillatoren
  - 24 bit Audio-CODEC mit Line- und Mikrofoneingängen und Line-Ausgang

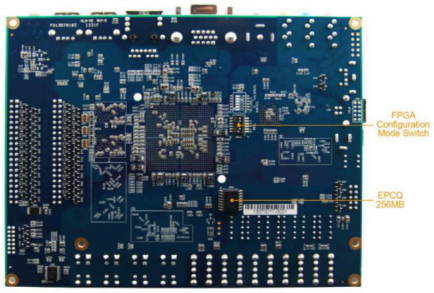
  - VGA DAC mit VGA-Ausgang
  - TV Decoder (NTSC/PAL/SECAM) und TV-Eingang
  - PS/2 Maus-/Tastaturanschluss
  - Infrarot Sender/Empfänger
  - Zwei 40-Pin GPIO-Leisten
- HPS
  - $800\,\mathrm{MHz}$ Dual-core ARM Cortex-A9 MPCore Prozessor
  - $1\,\mathrm{GB}$ DDR3 SDRAM ($32\,\mathrm{bit}$ Datenbus)
  - 1 Gigabit Ethernet PHY mit einem RJ45 Anschluss
  - 2-Port USB Host, normal Type-A USB Stecker
  - Micro SD-Kartenschacht
  - Beschleunigungssensor (I2C)
  - UART to USB, USB Mini-B Stecker
  - Warm-Reset und Cold-Reset-Taster
  - Ein benutzerdefinierte Taster, eine benutzerdefinierte LED
  - LTC 2x7 Erweiterungsstecker

In Abb. 2.3 ist die Ober- und Unterseite des DE1-SoC-Boards mit Hervorhebung der einzelnen Komponenten abgebildet.

Zu beachten ist die Einstellung des „FPGA Configuration Mode Switch“ auf der Unterseite des Boards (siehe Abb. 2.4). Davon hängt der Konfigurationszeitpunkt des FPGA und Programmierbarkeit des FPGA zur Laufzeit.



(a) Oberseite



(b) Unterseite

Abbildung 2.3.: DE1-SoC Board-Layout [5, S. 7]

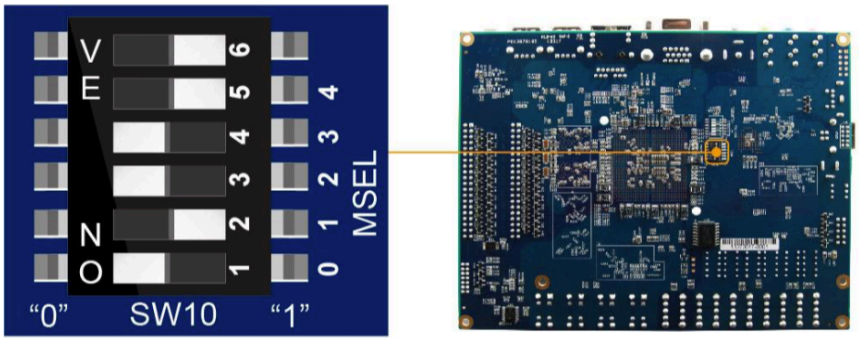


Abbildung 2.4.: FPGA Configuration Mode Switch [5, S. 12]

MSEL[4:0]	Konfiguration	FPGA-Konfigurationsquelle
10010	AS	EPCS (Voreinstellung)
01010	FPPx32	HPS-Software (Linux)
00000	FPPx16	HPS-Software (U-Boot, Abbild auf der SD-Karte)

Tabelle 2.1.: FPGA Konfigurationsmodi [5, S. 13]



Eine häufige Fehlerquelle ist eine spiegelverkehrte Einstellung der Konfigurationsschalter, wenn das Board so gehalten wird, dass die PCB-Legenden auf der Unterseite des Boards von links nach rechts gelesen werden können. Die Pin-Belegung ist nicht intuitiv, das MSB befindet sich **rechts**, und logisch „1“ ist **unten**

2.3. Quartus Prime 18.0

Um Hardware für einen FPGA entwickeln zu können, bedarf es einer passenden Entwicklungsumgebung. Da dem verwendeten Board ein Altera/Intel FPGA zugrundeliegt, wird die Entwicklungsumgebung „Quartus Prime“ verwendet. Diese ist nach einer Registrierung kostenfrei in der Version „Quartus Prime Lite Edition 18.0“ im Downloadportal[1] von Intel verfügbar.

In der Entwicklungsumgebung sind mehrere Anwendungen (siehe Abb. 2.5) enthalten, die in dieser Bachelorarbeit verwendet werden:

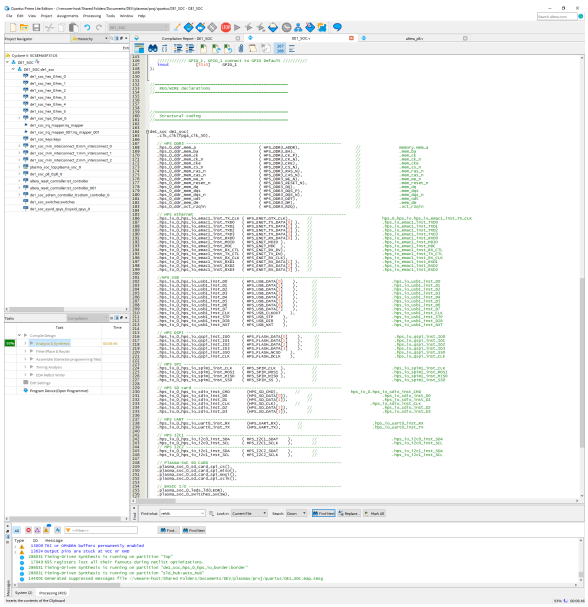
- Quartus Prime: Wird zur Hardware Definition Language (HDL)-Synthese verwendet
- Platform Designer: Grafisches SoC-Designwerkzeug
- Signal Tap: Synthetisierbarer Logikanalysator mit grafischer Benutzeroberfläche zur Signaldefinition und Auswertung
- Modelsim: HDL-Simulator

2.4. SoC FPGA Embedded Development Suite (SoC EDS)

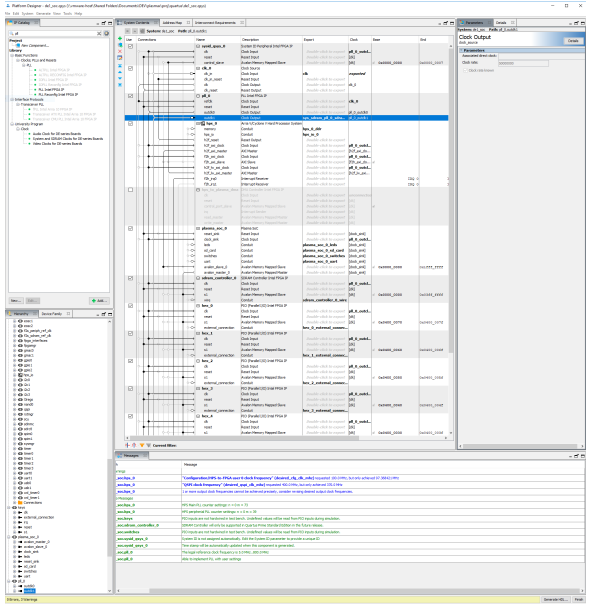
Um Software für ARM-basierte eingebettete Systeme entwickeln zu können, wird eine weitere Entwicklungsumgebung benötigt.

Genau so wie Quartus Prime ist SoC EDS nach einer Registrierung kostenfrei im Downloadportal[2] von Intel verfügbar.

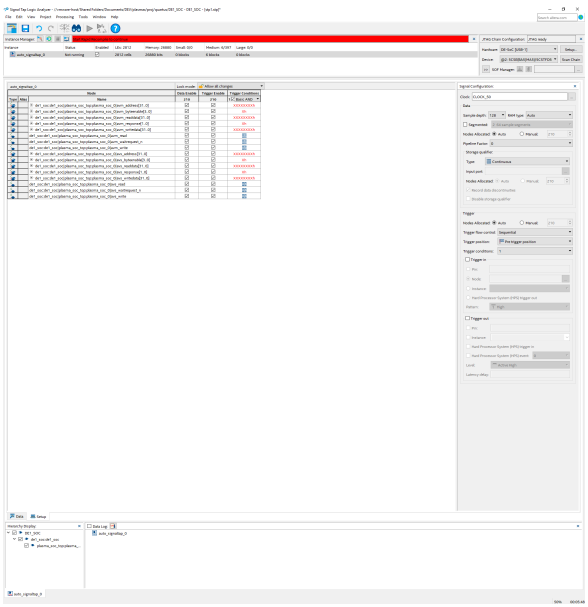
[1] `http://fpgasoftware.intel.com/?edition=lite`
[2] `http://fpgasoftware.intel.com/soceds/18.0/?edition=standard`



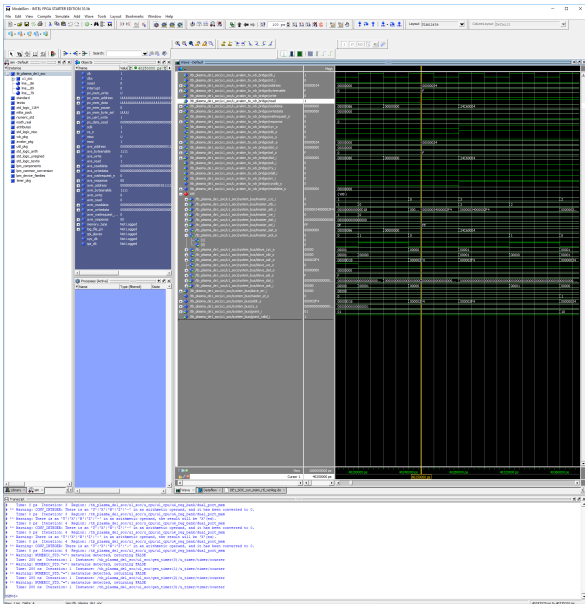
(a) Quartus



(b) Platform Designer



(c) Signal Tap



(d) Mentor Graphics Modelsim

Abbildung 2.5.: Quartus Anwendungen

Folgende Anwendungen werden in dieser Bachelorarbeit verwendet:

- Cross-Compiler für die ARM-Architektur
- Devicetree Compiler
- Board Support Package (BSP)-Editor

3. Grundlagen

In diesem Kapitel wird zunächst auf die Motivation für ein SoC und dessen Grundaufbau eingegangen. In diesem Zusammenhang wird auf die Bus-Architekturen „Wishbone“ und „Avalon“ eingegangen. Danach wird das Plasma-System von Steve Rhoads, sowie die Entwicklungs- und Evaluationsboards ZedBoard [3] von Digilent und DE1-SoC [5] von Terasic vorgestellt.

3.1. System-On-Chip

Der Begriff SoC wurde durch die Integration eines gesamten Systems (Prozessor, Speicher und Peripherie) auf einem einzigen Chip geprägt. Die SoCs sind aus modernen Geräten nicht wegzudenken. Ein prägnantes Beispiel hierfür ist ein Smartphone: auf einem einzigen Chip sind beispielsweise eine Central Processing Unit (CPU), Graphics Processing Unit (GPU), GSM-Modem, ein Display-Controller sowie der Kamera-Controller samt einem Digital Signal Processor (DSP) zur Bildverarbeitung integriert. Moderne Medizin-, Automobil-, Avionik-, Raumfahrt- und Militär-Systeme wären ohne SoCs nicht möglich. Doch auch banal erscheinende Geräte wie Küchengeräte und sogar die Heimbeleuchtung (Internet of Things (IoT) und Smart-Home) profitieren von der zunehmenden Miniaturisierung und den Einsatzmöglichkeiten der SoCs.

Der Entwurf eines SoC ist mit einem sehr hohen Zeit- und Arbeitsaufwand verbunden. Je komplexer das System, desto aufwändiger wird ein von Grund auf neuer Aufbau. [6, S. 189] Darum wird heutzutage, analog zur Softwareentwicklung, auf erprobte, wiederverwendbare Komponenten gesetzt. Diese sind als IP-Cores verfügbar und können in eigene Projekte integriert werden. Davon sind einige parametrisierbar und somit auf Projektbedürfnisse anpassbar. Ein IP-Core kann als high-level HDL Code, auch Soft-Core genannt, oder auch als ein detailliertes Layout auf Transistorebene, Hard-Core genannt, ausgeliefert werden. [6, S. 189]

Ein gutes Beispiel für parametrisierbare IP-Cores ist die ARM-CPU. ARM als Unternehmen ist kein Prozessorhersteller an sich, sondern bietet ledeglich die CPU-Architektur als IP-Core zur Erwerb an. Ein Interessent kann diese lizenzieren, auf

seine Bedürfnisse anpassen und bei einem Halbleiterfertiger in Produktion geben. Durch Kombination verschiedener auf dem Markt verfügbarer IP-Cores kann innerhalb kürzester Zeit ein neues Produkt mit einem gegenüber den Einzelkomponenten erhöhten Mehrwert entwickelt werden.

3.2. Bussysteme

In vorigen Abschnitten wurden beispielhaft einige Komponenten eines SoC genannt. Im Wesentlichen handelt es sich hierbei um eine CPU, die mit sämtlichen domänenspezifischen Prozessoren und Peripheriegeräten bzw. Schnittstellen kommuniziert. Essentiell hierfür ist ein Bus-System, welches allen relevanten Modulen ein Kommunikationsmedium bereitstellt. Verbreitet sind standardisierte Bus-Architekturen wie AMBA, CoreConnect, Wishbone, STBus und weitere. [7, S. 2] AMBA ist sehr verbreitet in der Entwicklung von SoCs auf Basis von ARM-Prozessoren. In dieser Arbeit wird besondere Aufmerksamkeit dem Wishbone- und Avalon-Bus gewidmet.

3.2.1. Topologien

Unter einer Bus-Topologie versteht man die Anordnung der Master- und Slavegeräte zueinander. Je nach Topologie muss u.U. eine Arbitrierung vorgenommen werden, um Ressourcen unter mehreren Teilnehmern verteilen zu können. Die nachfolgenden Abbildungen entstammen der Wishbone-Spezifikation [8] und sind auch auf andere Bussysteme übertragbar.

In Abb. 3.1 ist eine Crossbar-Switch-Topologie dargestellt. In diesem Fall können mehrere Master zeitgleich eine Verbindung zu einem Slave aufnehmen. Jede Verbindung ist unabhängiig von den anderen und kann parallel betrieben werden. Durch diese Parallelität erhöht sich der Durchsatz des ganzen Systems. Diese Topologie ist geschwindigkeitsmäßig den klassischen Bussystemen überlegen. Grundsätzlich unterstützen die Crossbar Routingmechanismen dynamische Konfiguration, was zu einen zuverlässigem und konfigurierbaren Netzwerk führt. Außerdem sind die meisten Crossbar-Architekturen skalierbar, können also bei Bedarf erweitert werden. [8, S. 16]

In Abb. 3.2 ist eine Data-Flow-Topologie dargestellt. Hierbei fließt der Datenstrom sequentiell durch einen vordefinierten Satz an IP-Cores. Data-Flow-Architekturen weisen oft eine Parallelität auf: eine oder mehrere werden Funktionen zeitgleich ausgeführt. [8, S. 17]

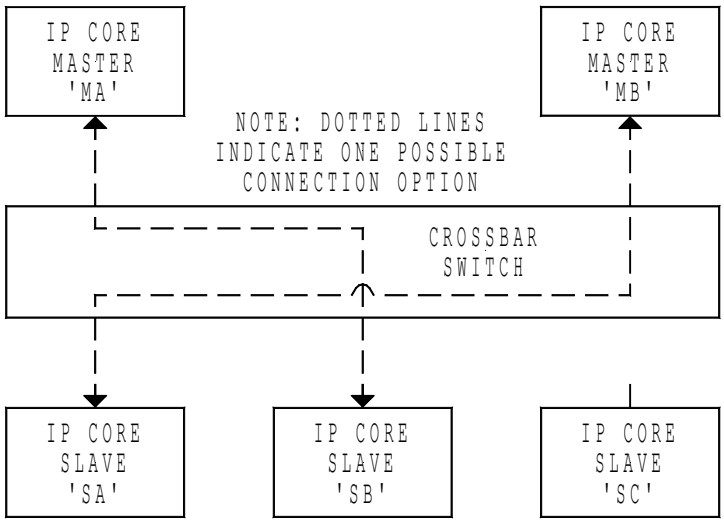


Abbildung 3.1.: Crossbar (Switch) [8, S. 17]

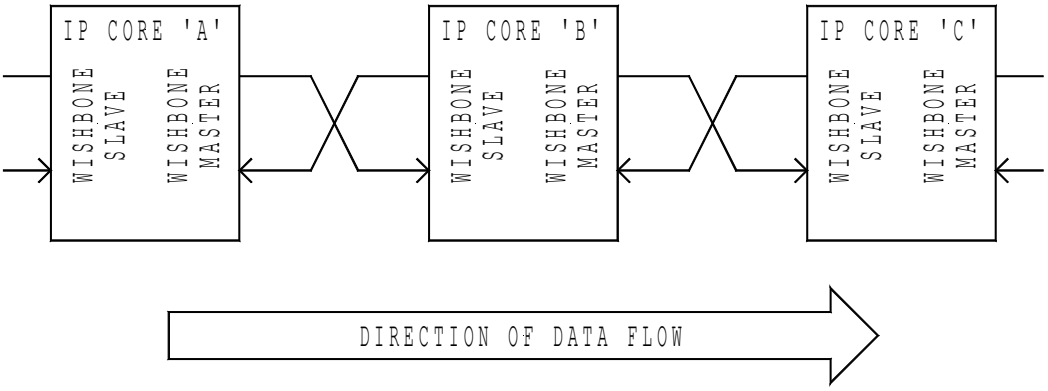


Abbildung 3.2.: Data flow [8, S. 17]

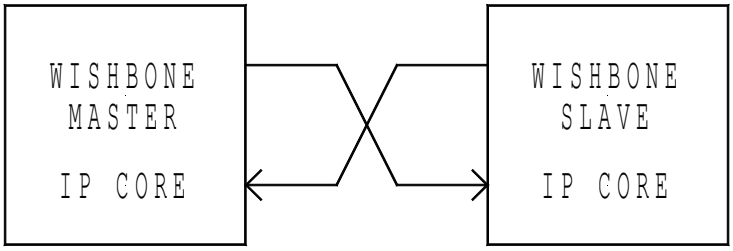


Abbildung 3.3.: Point to point [8, S. 21]

Eine Punkt-zu-Punkt Topologie, wie in Abb. 3.3 abgebildet, ist die einfachste aller Topologien mit einem einzigen Master und einem Slave.[8, S. 20]

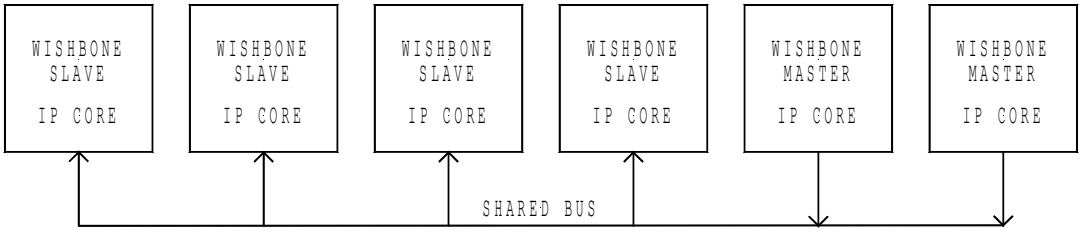


Abbildung 3.4.: Shared bus [8, S. 21]

In Abb. 3.4 ist eine Shared-Bus-Topologie dargestellt. Hierbei initiiert ein Master einen Buszyklus zum Slave. Traditionelle Busse wie VMEbus und PCI Bus nutzen diese Topologie. Architekturbedingt kann hier nur ein Master zu einem Zeitpunkt den Bus nutzen. [8, S. 21]

3.2.2. Wishbone

Wishbone ist ein Open-Source-Hardware Computerbus, welcher die Kommunikation der Komponenten eines SoC durch eine einheitliche Schnittstelle deutlich vereinfacht. Entwickelt wurde dieser von der Silicore Corporation, und später von OpenCores[1] veröffentlicht. Überwiegend wird Wishbone-Standard von quelloffenen Hardware-Projekten verwendet.

Eine mögliche Portkonfiguration für Wishbone-Master bzw Slave ist in Abb. 3.5 dargestellt.

3.2.3. Avalon

Avalon ist eine proprietäre Bus-Architektur und wurde ursprünglich von Altera entwickelt, um On-Chip-Prozessoren und Peripherie zu einem System-on-a-programmable-chip (SOPC) zu verknüpfen. [7, S. 7–6] Nutzbar ist Avalon als eine Sammlung parametrisierbarer IP-Cores, die vom Platform Designer (früher als Qsys bzw. SOPC Builder bekannt) im Laufe der Systementwicklung automatisch generiert und parametrisert werden.

Ein mögliches System mit Avalon ist auf Abb. 3.6 abgebildet.

[1] `https://opencores.org`

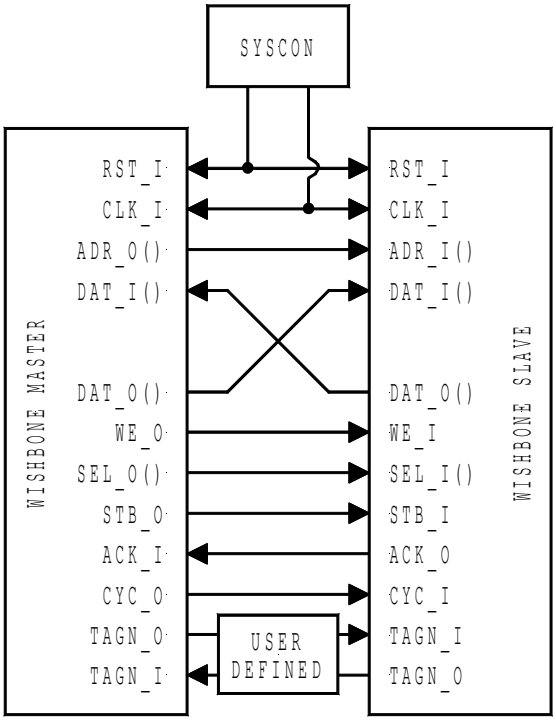


Abbildung 3.5.: Standard connection for timing diagrams [8, S. 14]

Die Avalon-Spezifikation sieht sieben Schnittstellentypen vor [9, S. 4]:

- Avalon Streaming Interface (Avalon-ST) - unterstützt unidirektionalen Datenfluss (vgl. Abb. 3.2)
- Avalon Memory Mapped Interface (Avalon-MM) - unterstützt adressbasierte bidiriektionale Master-Slave Kommunikation
- Avalon Conduit Interface - fasst Signale, die nicht einem Avalon-Schnittstellentyp zuzuordnen sind, zusammen. Eine Verbindung im Platform Designer bzw. Export zur manuellen Verknüpfung ist möglich
- Avalon Tri-State Conduit Interface (Avalon-TC) - unterstützt Verbindungen zu Off-Chip Komponenten. Mehrere Peripheriegeräte können Pins durch Signal-multiplexing teilen
- Avalon Interrupt Interface - ermöglicht Komponenten Ereignisse an andere Komponenten zu vermitteln
- Avalon Clock Interface - Schnittstelle für Taktein- bzw. Ausgang

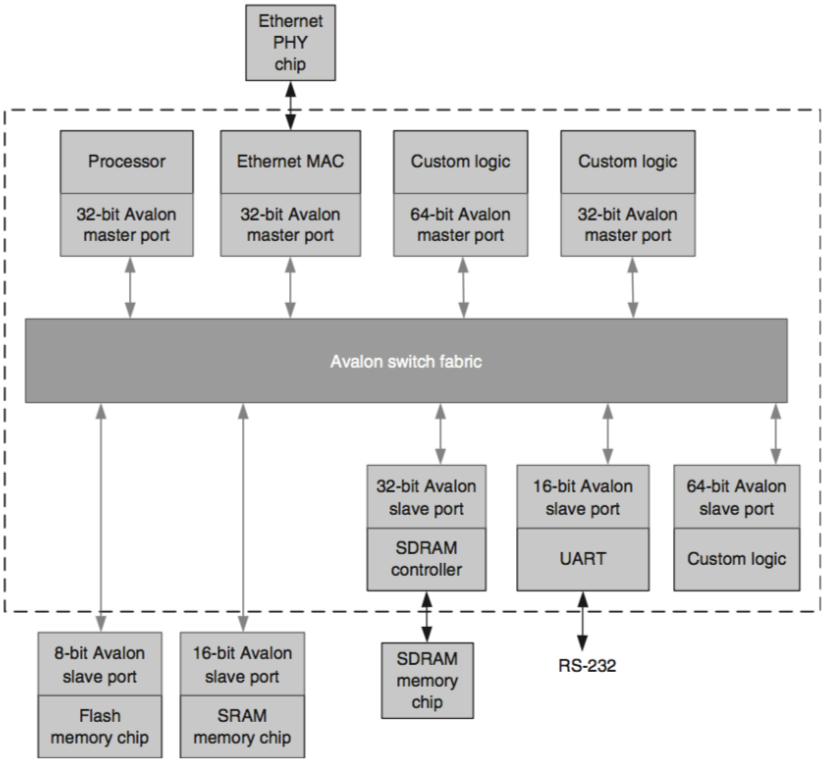


Abbildung 3.6.: Avalon-Bus basiertes System [7, S. 7–6]

- Avalon Reset Interface - Schnittstelle für Reset-Logik

Avalon nutzt separate Adress-, Daten-, und Steuerleitungen. [7, S. 7–6] In Tabelle 3.1 sind die für diese Arbeit wichtigsten Avalon-MM Signale aufgelistet.

Signal	**Beschreibung**
address	Adresse
byteenable	Byte-Selektor
read	Signalisiert einen Lesezugriff
write	Signalisiert einen Schreibzugriff
readdata	Vom Slave übertragenes Datenwort
writedata	An den Slave übertragenes Datenwort
waitrequest	An den Master übermitteltes Wartesignal
response	Optionaler Antwortcode

Tabelle 3.1.: Avalon-MM Signale [9, S. 14–16]

3.3. Plasma CPU

Die Plasma CPU ist ein kleiner, synthetisierbarer 32-bit Reduced Instruction Set Computer (RISC) Mikroprozessor. Derzeit führt es einen Webserver mit einem Interruptcontroller, Universal Asynchronous Receiver-Transmitter (UART), Static Random Access Memory (SRAM) oder DDR SDRAM Controller, und einem Ethernetcontroller aus. Die Plasma CPU führt bis auf unausgerichtete Lade und Speicheroperationen alle Microprocessor without Interlocked Pipeline Stages (MIPS) I User-Mode Instruktionen aus [1].

Der auf dem Plasma ausgeführte Webserver ist unter der angegebenen Adresse *https://plasmacpu.no-ip.org:8080/* [1], geprüft am 22. April 2018, nicht erreichbar.

In Abb. 3.7 ist das Block-Diagramm der Plasma-CPU dargestellt.

3.4. Device Tree

Der Device Tree ist eine baumartige Datenstruktur, mit der eine deklarative Beschreibung der auf einem System verfügbaren Hardware möglich ist. Jeder Knoten hat Schlüssel/Wert-Paare, welche die Charakteristika des repräsentierten Geräts beschreiben. Die Baum-Struktur ist dadurch gegeben, dass jeder Knoten genau einen

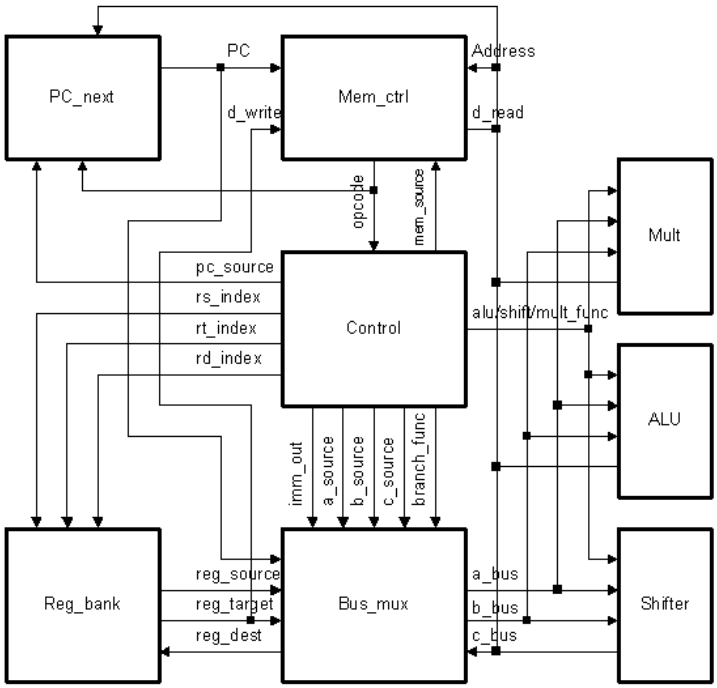


Abbildung 3.7.: Plasma-CPU Blockdiagramm [1]

Elternknoten besitzt. Die einzige Ausnahme ist der Wurzelknoten, welcher keinem Elternknoten zugeordnet ist [10, S. 6].

Bevor die Device Tree-Spezifikation eingeführt wurde, beinhaltete der Linux-Kernel gerätespezifischen Code. Bereits geringfügige Änderungen, wie die Änderung einer Basisadresse, führten dazu, dass der ganze Kernel neu kompiliert werden musste [11, S. 1].

Für eingebettete Systeme, welche im Gegensatz zu größeren Systemen wie Desktop-Computern nur selten über automatische Erkennungsmechanismen neuer Hardware verfügen, bedeutet es einen enormen Entwicklungs- und Wartungsaufwand. Dieser Aufwand übertrug sich auch auf die Kernel-Entwickler, die Unmengen an verschiedensten Systemkonfigurationen im Kernel verwalten mussten.

3.5. Lebenszyklus des Cyclone V SoCFPGA

Am Startvorgang eines komplexen Rechnersystems sind etappenweise mehrere Softwaresysteme beteiligt. So kann die Firmware die initiale Hardwarekonfiguration vornehmen, um dann die Kontrolle an eine weitere Software wie z.B einen Bootloader

oder ein Betriebssystem zu übergeben [10, S. 3]. Das Standard-Szenario für Altera SoCFPGA-Systeme ist in Abb. 3.8 abgebildet.

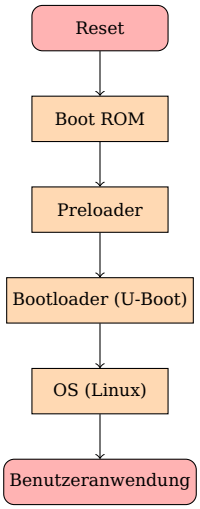


Abbildung 3.8.: Bootvorgang [12, S. 95]

Der erste Bootloader befindet sich im BootROM und ist fest vom Hersteller Altera einprogrammiert. Seine Aufgabe besteht darin, den Master Boot Record (MBR)-Eintrag der Speicherkarte auszulesen, die Altera-spezifische Partition mit der Kennung `0xA2` zu identifizieren und den Preloader vom Anfang der Partition zu verifizieren und auszuführen [12, S. 98].

Die Aufgabe des Preloaders besteht darin, sämtliche Taktgeneratoren, phase locked loops (PLLs), RAMs, Input/Outputs (IOs) und Pin-Multiplexer zu initialisieren. Im Anschluss wird vom Preloader von der Partition mit der Kennung `0xA2` der nächste Bootloader, wie z.B. „Das U-Boot“, verifiziert, geladen und gestartet.

Werden die HPS-Peripherieeinstellungen im Platform Designer verändert, so muss der Preloader neu generiert, kompiliert und auf das Startmedium übertragen werden [12, S. 96].



Eine häufige Fehlerquelle ist ein veralteter Preloader. Dieser muss bei jeder Änderung der HPS-Konfiguration im Platform Designer aktualisiert werden! [12, S. 96]

Für eingebettete Systeme eignet sich der Bootloader „Das U-Boot“. Dieser Bootloader ist quelloffen, portabel, bietet Treiber für ein breites Spektrum an Peripheriegeräten und richtet eine Umgebung mit einer Kommandozeile ein. Außerdem lädt er

den Device Tree und ist in der Lage, diesen zur Laufzeit mit aktuellen Daten zu modifizieren. Sobald die Umgebung eingerichtet ist, lädt dieser Bootloader den Linux-Kernel vom Dateisystem, Netzwerk oder serieller Schnittstelle und übergibt sämtliche Bootargumente an den Kernel [12, S. 105–106].

4. Systemarchitektur

Dieses Kapitel beschreibt die vorhandene Architektur, schlägt neue vor und dient dem besseren Verständnis über die Umsetzung in Kapitel 5.

Im Abschnitt 4.1 wird die Architektur des Ausgangssystems in der Version 2.1 beschrieben.

Im Abschnitt 4.2 wird eine neue Architektur, die den Wishbone-Interconnect nutzt, neue Komponenten aufweist und speziell auf das Zedboard zugeschnitten ist, vorgestellt.

Im Abschnitt 4.3 werden zwei neue Architekturen vorgeschlagen. Die erste stellt einer Portierung auf das DE1-SoC Board dar, in der zweiten wird die erste Architektur erneut aufgegriffen und erweitert.

4.1. Plasma

Auf Abb. 4.1 ist die Architektur des Plasma-Systems im Ausgangszusatand (Version 2.1) abgebildet. Diese beinhaltet nur einen rudimentären Interrupt-Controller, zwei General-Purpose Input/Outputs (GPIOs), einen Counter sowie einen UART Controller. Außerdem ist der interne RAM als FPGA-interner Block-RAM implementiert und der externe RAM ungenutzt. Die Verknüpfung zwischen der Plasma-CPU und den Komponenten erfolgt über einen sehr einfachen Multiplexer mit Adressdekoder, dessen Erweiterung sich allerdings als äußerst umständlich erweist.

4.2. Plasma-SoC auf dem ZedBoard

Die erste Entwicklungsiteration erfolgt für das Digilent ZedBoard. Es werden ein Wishbone-Interconnect und die dazugehörigen Wrapper bzw. Adapter entwickelt. Außerdem werden neue Komponenten wie z.B. Zähler, ein Controller für das Display mit der Modellnummer UG-2832HSWEG04 sowie ein Serial Peripheral Interface

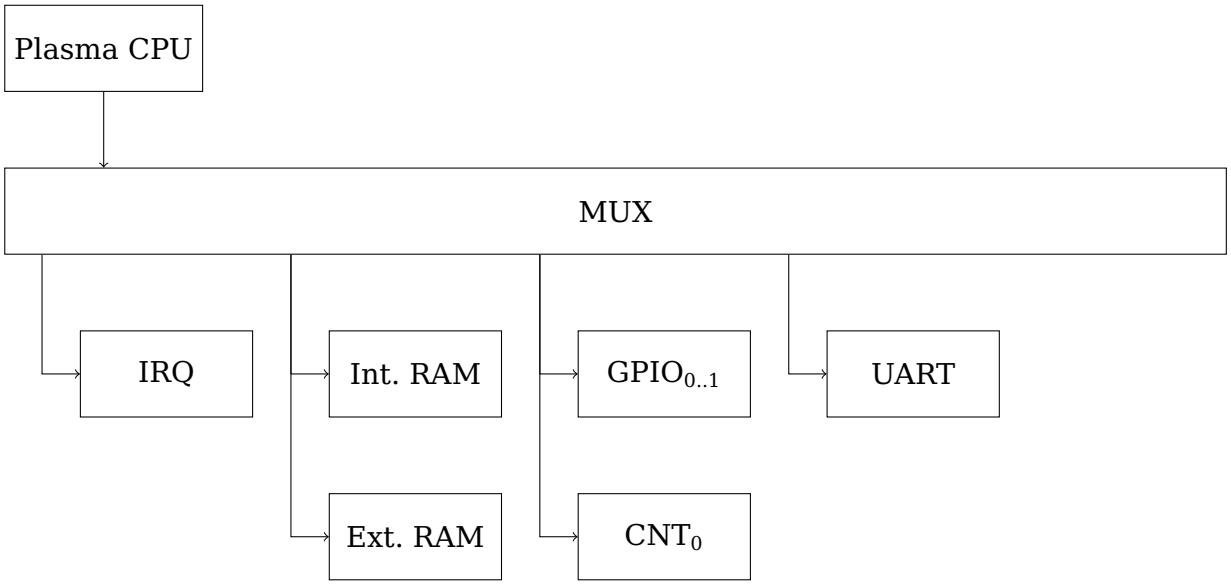


Abbildung 4.1.: Blockdiagramm des Plasma im Ausgangszustand

bus (SPI)-Controller eingeführt. Einen Überblick über die neue Systemarchitektur verschafft Abb. 4.2.

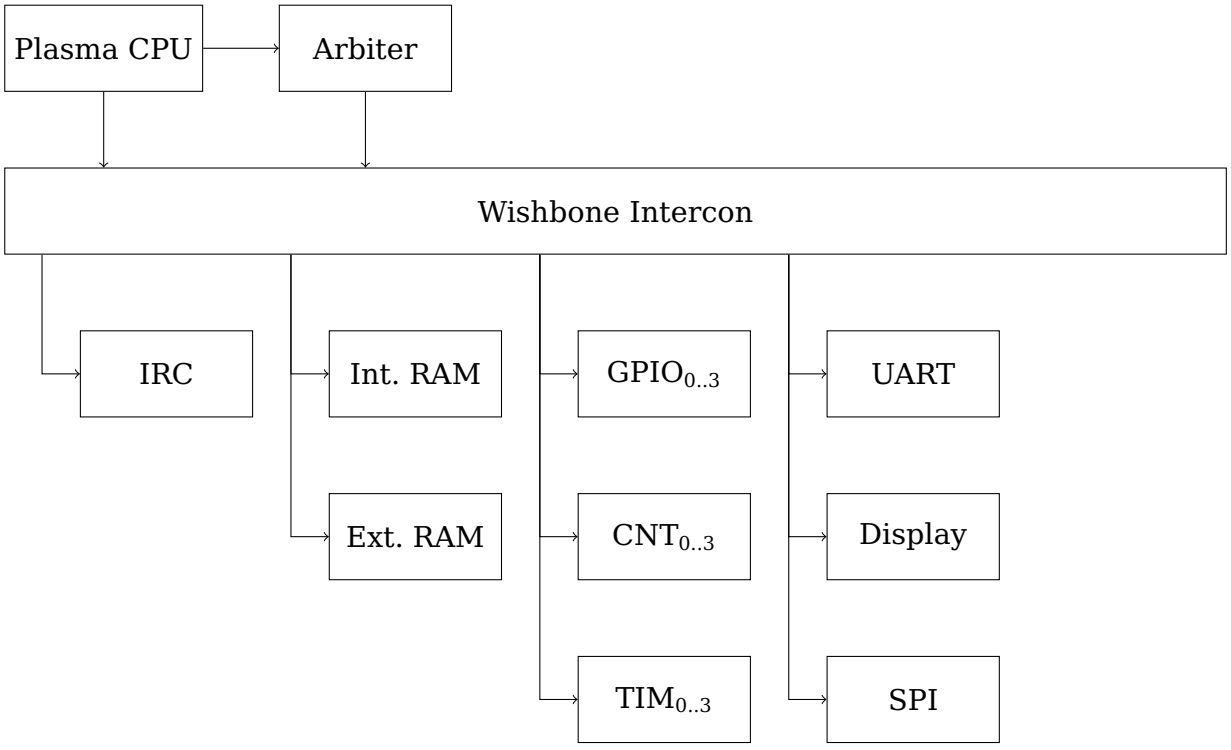


Abbildung 4.2.: Blockdiagramm des Plasma-SoC auf dem ZedBoard

4.3. Plasma auf dem DE1-SoC-Board

In der zweiten Iteration wird das vorhandene Plasma-SoC auf das neue Terasic DE1-SoC-Board portiert. Die dabei verfolgte Architektur ist auf Abb. 4.3 abgebildet. Da das Terasic DE1-SoC-Board nicht über ein Display verfügt, wird dieser aus dem Wishbone-Bus und der Adresszuordnung entfernt. Ebenso wird auf die externe RAM-Schnittstelle verzichtet.

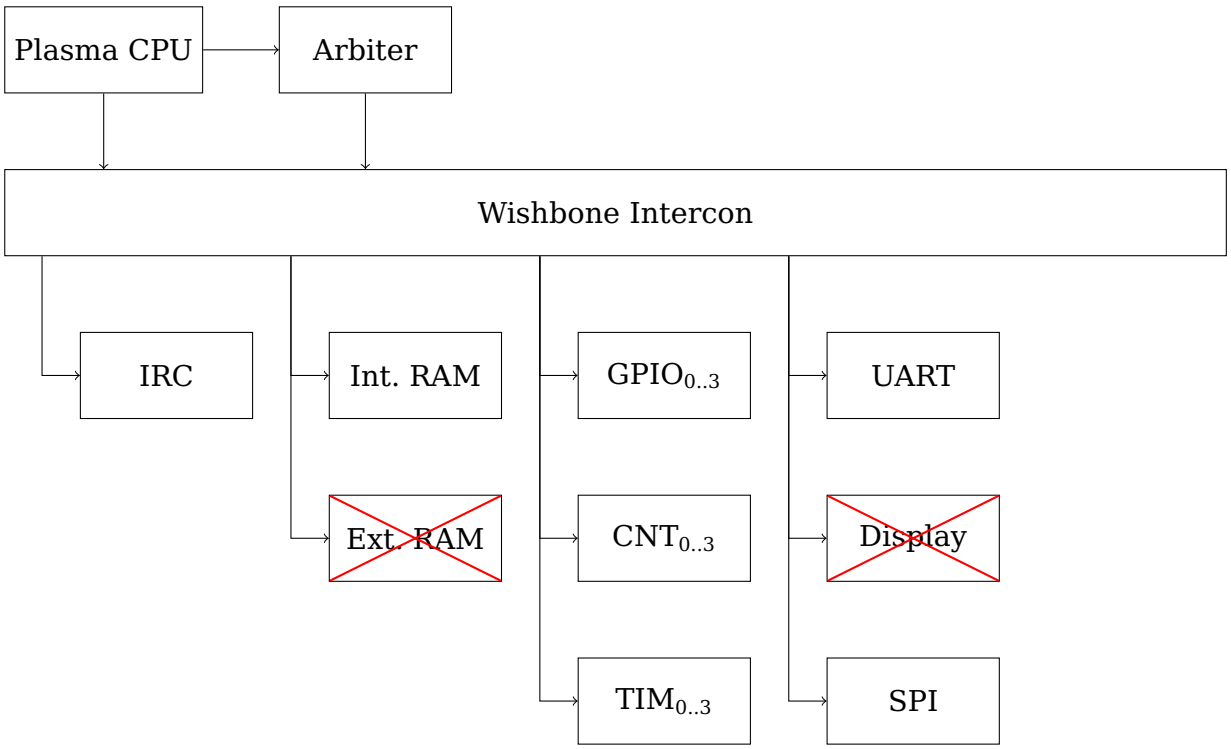


Abbildung 4.3.: Blockdiagramm des Plasma-SoC auf dem DE1-SoC

Die dritte Iteration sieht eine Erweiterung des auf das Terasic DE1-SoC-Board portierten Plasma-SoC vor. In diesem Schritt werden jeweils eine Avalon-Wishbone und eine Wishbone-Avalon Bridge als Master bzw. Slave zum Wishbone-Bus hinzugefügt, wie auf Abb. 4.4 dargestellt. Diese Neuerung und die Erstellung eines IP-Core ermöglicht eine Integration in das das im Platform Designer entworfene QSYS-System.

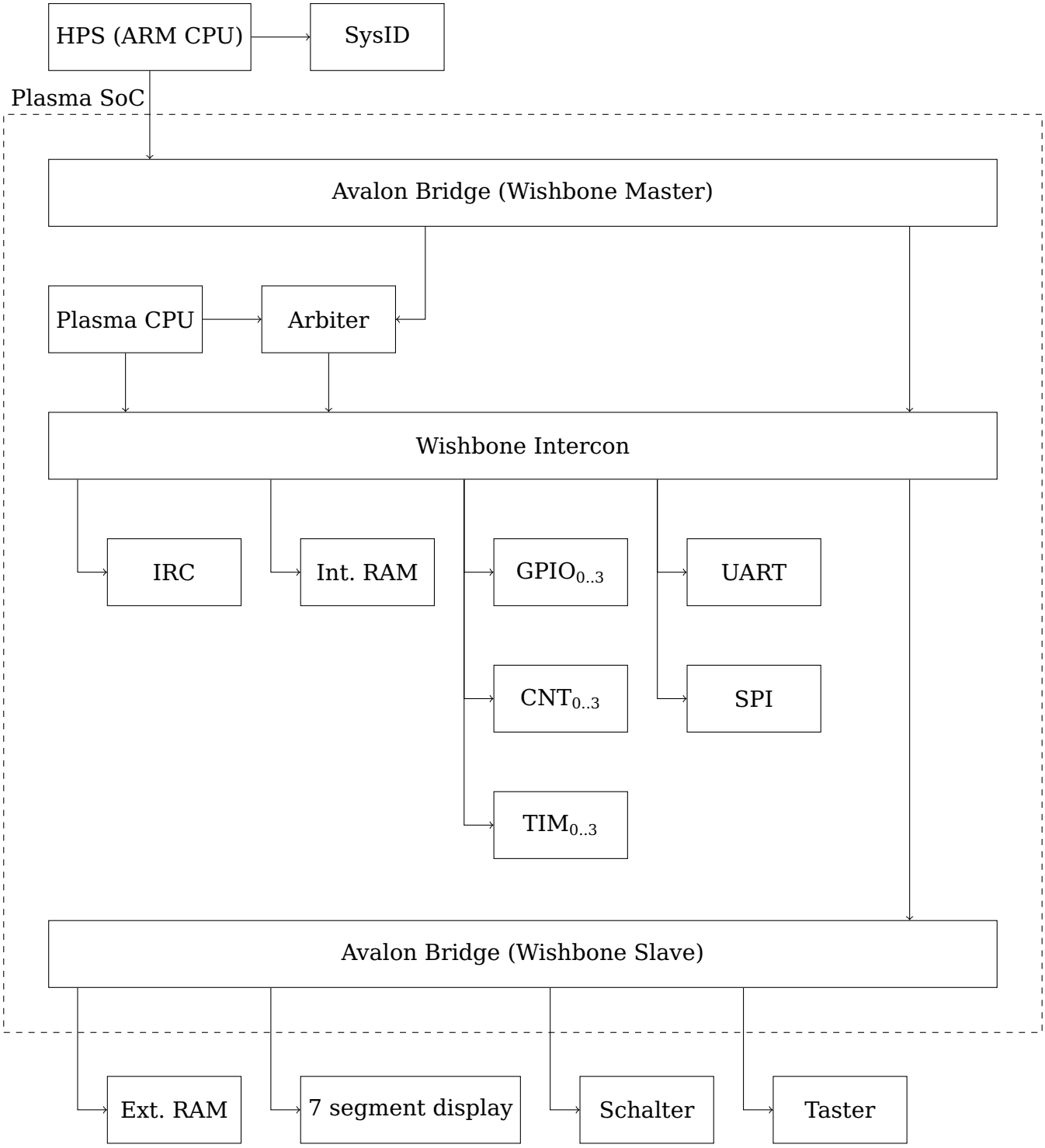


Abbildung 4.4.: Blockdiagramm des Plasma-SoC auf dem DE1-SoC mit Verbindung zu weiteren IP-Cores und ARM Prozessor über den Avalon-Bus

5. Umsetzung

In diesem Kapitel wird auf die Umsetzungsdetails der Hardware- und Softwareentwicklung, sowie der Systemeinrichtung eingegangen.

Im Abschnitt 5.1 werden wesentliche Etappen der Entwicklung des Wisbone-Interconnect, neuer Module und Wrapper für bereits vorhandene Komponenten geschildert, um nachvolgende Erweiterungen besser nachvollziehen zu können.

Die Entwicklung der Firmware für das Plasma-SoC wird in Abschnitt 5.2 umrissen.

Auf die Portierung des im Abschnitt 5.1 geschilderten SoC auf das in dieser Bachelorarbeit verwendete DE1-SoC Board wird in Abschnitt 5.3 eingegangen.

Im Abschnitt 5.4 wird das System aus Abschnitt 5.3 um zusätzliche Hardware-Komponenten erweitert, sodass der externe, $64\,\mathrm{MB}$ umfassende, RAM und auch die ARM-CPU verwendet werden können.

Im Abschnitt 5.5 wird die Erweiterung der Hilfssoftware geschildert.

Die Portierung der Plasma-Firmware und der Plasma-Beispielanwendung auf das DE1-SoC wird im Abschnitt 5.6 geschildert.

Der Abschnitt 5.7 beschreibt die Einrichtung eines Linux-Betriebssystems auf dem DE1-SoC Board und der dafür notwendigen Schritte.

Im Abschnitt 5.8 wird die Entwicklung der Linux Anwendungen, welche mit auf dem HPS ausgeführt werden und Zugriff auf das Plasma-SoC haben, beschrieben.

Im Abschnitt 5.9 wird die Entwicklung der Linux Treiber, welche einen alternativen Zugriff auf das Plasma-Soc erlauben, beschrieben.

5.1. Entwicklung des Wishbone-basierten Systems

Im Ausgangssystem stehen alle Komponenten durch manuelle Adressdekodierung und Datenmultiplex dem Plasma zur Verfügung. Jegliche Erweiterung erforderte

mehrere Eingriffe und Komponenten mit mehreren Kontroll- und Datenregistern waren nur sehr schwer einzubinden.

Es wird ein Shared-Bus (vgl. Abb. 3.4) gemäß der Wishbone-Spezifikation [8, S. 21] entwickelt. Als Entwicklungsgrundlage dient das „boschmitt/wishbone“[13] Projekt.

Sollte eine Adresse außerhalb des gültigen Bereiches angesprochen werden, so signalisiert der Bus einen Busfehler. Ein Fehler wird ebenfalls signalisiert, wenn eine korrekt ausgewählte Komponente innerhalb ihrer eigenen Adressdekodierung eine Zugriffsverletzung feststellt. Um einen Busfehler an den Kernel weiterzugeben, wird der Interrupt 31 verwendet (vgl. Tabelle 5.4).

Die Adressbereiche, welche bei der partiellen Adressdekodierung verwendet werden, sind in Tabelle 5.1 aufgelistet.

Gerät	**Startadresse**	**Adressbereich**
Interner RAM	0x00000000	0x0FFFFFFF
Externer RAM	0x10000000	0x001FFFFF
IRC_0	0x20000000	0x1F
$UART_0$	0x20000100	0x0F
$Timer_{0:3}$	0x20000200	0x0F
	0x20000210	0x0F
	0x20000220	0x0F
	0x20000230	0x0F
$Zähler_{0:3}$	0x20000300	0x0F
	0x20000310	0x0F
	0x20000320	0x0F
	0x20000330	0x0F
$GPIO_{0:3}$	0x20000400	0x0F
	0x20000410	0x0F
	0x20000420	0x0F
	0x20000430	0x0F
SPI_0	0x20000500	0x0F
Display	0x40000000	0x7F

Tabelle 5.1.: Adresszuordnung

Um destruktive Änderungen an bereits vorhandenen Komponenten, wie der Plasma-CPU, RAM, UART, etc. zu vermeiden, werden Wishbone-Wrapper als zusätzliche Entitäten entwickelt. Dieses Entwurfsmuster wird auch für neue Module verwendet, da sie zum Teil auch als Untermodule in anderen Komponenten verwendet werden.

Zu den neu eingeführten Komponenten gehören der Interrupt-Controller, Zähler, Timer, GPIO-, UART-, SPI-Schnittstellen sowie der Display-Controller.

Es wird ein einheitliches Register-Schema für alle Komponenten entworfen (siehe Tabelle 5.2). An den unteren drei Adressen liegen nacheinander das Kontroll-, Status- und Datenregister. Die jeweils unteren beiden Bits sind für Control/Status Register (CSR) reserviert.

Offset	**Register**	**R/W**	**31 .. 2**	**1**	**0**
0x00	control	r/w	user defined	enable	reset
0x04	status	r	user defined	data available	ready
0x08	data	r/w	data		
⋮	⋮	⋮	⋮		

Tabelle 5.2.: Allgemeine Register-Map

Der Plasma Prozessor wird an das Wishbone-Interface angepasst. Dazu wird ein Wishbone-Master-Wrapper entwickelt. Dieser kontrolliert die *cyc* und *stb* Ausgangssignale, und reagiert auf *ack*, *stall*, *err*, *rty* Signale.

Der interne RAM wird an den Wishbone Bus angebunden. Der entsprechende Wrapper ist im Vergleich zu anderen Devices sehr einfach, da das in Tabelle 5.2 beschriebene Schema hier nicht befolgt werden muss und die Daten ohne weitere Adressdekodierung am Bus anliegen.

Der Interrupt Controller wird aus P. Tofahrns Abschlussprojekt [14] übernommen und angepasst. Für diesen wird ein Wishbone-Wrapper entwickelt. Zusätzlich waren Änderungen an der Plasma-CPU notwendig, da die CPU „bei einem Reset an Adresse 0 und bei einem Interrupt an 0x3C“ [14, S. 5] sprang.

Für sämtliche Komponenten werden Interrupts definiert. Da der neue Interrupt-Controller mit 32 Kanälen parametrisiert ist, können 32 verschiedene IRQs definiert werden. Diese sind in Tabelle 5.4 aufgelistet. Dabei sind die IRQs 30 bis 15 nicht zugeordnet und für künftige Erweiterungen gedacht.

Ein neuer Zähler mit neuen Features wird entwickelt. Bei diesem sind die Zählerrichtung, Reload-Wert sowie der Auto-Reload zur Laufzeit konfigurierbar. Bei Erreichen eines Grenzwertes (Aufwärts: reload - 1, Abwärts: 0) wird ein Interrupt ausgelöst. Die Register-Map ist in Tabelle 5.5 dargestellt.

Die Timer-Komponente baut auf dem obengenannten Zähler auf und erbt dessen wesentlichen Eigenschaften. Der Reload-Wert wird intern - abhängig von der konfigu-

Offset	Register	R/W	31 .. 2	1	0
0x00	control	r/w		enable	reset
0x04	status	r	unused		
0x08	data	r/w	data		
0x0C	imm flags	r	immediate interrupt flag state		
0x10	clear	r/w	clear interrupt flag		
0x14	invert	r/w	polarity		
0x18	mask	r/w	interrupt mask		
0x1C	edge	r/w	edge detection		

Tabelle 5.3.: Interrupt Controller Register-Map

IRQ	31	30:15	14	13:10	9:6	5:2	1	0
	Busfehler	$\emptyset$	Display	$GPIO_{3:0}$	$Zähler_{3:0}$	$Timer_{3:0}$	$UART_w$	$UART_r$

Tabelle 5.4.: IRQs

Offset	Register	R/W	31 .. 4	3	2	1	0
0x00	control	r/w		autoreload	direction	enable	reset
0x04	status	r				overflow	ready
0x08	data	r	current counter value				
0x0C	reload	r/w	reload value				

Tabelle 5.5.: Zähler Register-Map

rierten Zeiteinheit - modifiziert. Bei Erreichen des Schwellwertes wird analog zum Zähler ein Interrupt ausgelöst.

Offset	**Register**	**R/W**	**31 .. 6**	**5:3**	**2**	**1**	**0**
0x00	control	r/w		unit (1)	autoreload	enable	reset
0x04	status	r				overflow	ready
0x08	data	r	current timer value				
0x0C	reload	r/w	reload value				
Anmerkungen: 1. Erlaubte Zeiteinheiten sind aus Tabelle 5.7 zu entnehmen							

Tabelle 5.6.: Timer Register-Map

Einheit	**Code**	**Reload-Wert**
$1\,\mathrm{tick}$	0	1
$1\,\mathrm{ns}$	1	$50 * 10^{-3}$
$1\,\mu\mathrm{s}$	2	$50 * 10^{0}$
$1\,\mathrm{ms}$	3	$50 * 10^{3}$
$1\,\mathrm{s}$	4	$50 * 10^{6}$

Tabelle 5.7.: Timer Zeiteinheiten Kodierung und Reloadwert [ticks]

Für den bereits im Plasma-System vorhandenen UART-Controller wird ein Wishbone-Wrapper entwickelt. Die zugrundeliegende Komponente wird fest für eine Systemfrequenz von $50\,\mathrm{MHz}$ konfiguriert, sodass eine Kommunikation mit $57\,600\,\mathrm{Bd}$ erfolgen kann.

Offset	**Register**	**R/W**	**31 .. 2**	**1**	**0**
0x00	control	r/w		enable	reset
0x04	status	r		data available	ready
0x08	data	r/w	data		

Tabelle 5.8.: UART-Controller Register-Map

Um Peripherie über GPIO ansteuern zu können, wird ein rudimentärer GPIO-Controller entwickelt. Dieser unterstützt einfache Ausgaben; die Eingaben sind jedoch über eine konfigurierbare Maske maskiert, sodass nur bei bestimmten Pins ein Interrupt ausgelöst wird.

Offset	Register	R/W	31 .. 2	1	0
0x00	control	r/w		enable	reset
0x04	status	r		data available	ready
0x08	data	r/w	data		
0x0C	mask	r/w	mask		

Tabelle 5.9.: GPIO-Controller Register-Map

Basierend auf den SPI Controllern aus den „ZedBoard OLED Display“ [15] und „NexysVideo/GPIO“ [16] Projekten wird ein generischer SPI-Master entwickelt. Konfigurierbar sind die Anzahl Slaves, Datenwortbreite, System- und SPI-Frequenz. Aus der System- und SPI-Frequenz wird der Faktor für den Frequenzteiler gewonnen.

Offset	Register	R/W	31 .. 2	1	0
0x00	control	r/w		enable	reset
0x04	status	r		overflow	ready
0x08	data	r/w	data		
0x0C	address	r/w	SPI slave address		

Tabelle 5.10.: SPI Controller Register-Map

Aufbauend auf Michael Mattioli’s Projekt „ZedBoard OLED Display“ [15] wird der Display-Controller entwickelt. Der Display-Controller unterstützt derzeit nur den Textmodus, d.h der Inhalt des VRAM wird als ASCII-Zeichen interpretiert. Weitere Details sind der Registerzuordnung in Tabelle 5.11 zu entnehmen.

Offset	Register	R/W	31 .. 6	5	4	3	2	1	0
0x00	control	r/w		clear	flush	text	imm	enable	reset
0x04	status	r							ready
0x08:0x208	data	w	data						

Tabelle 5.11.: Display-Controller Register-Map

5.2. Entwicklung der Plasma-Firmware

Zur Inbetriebnahme des Plasma-SoC benötigt man eine Firmware. Die neu entwickelten Geräte erfordern Treiber, um von der Software angesteuert werden zu können.

Bit	Name	Beschreibung
0	reset	Setzt das Display zurück
1	enable	Aktiviert das Display
2	imm	Datenregister wird als Display-Commandos interpretiert
3	text	VRAM wird als ASCII-Zeichen interpretiert
4	flush	Schreibt das VRAM an das Display
5	clear	Leert den VRAM

Tabelle 5.12.: Control Register Bits

5.2.1. Treiber

Die entwickelten Treiber bauen auf memory-mapped Strukturen auf. Dies hat den Vorteil, dass man nicht jede einzelne Speicher- bzw. Geräteadresse über eine Basisadresse+Offset ansprechen muss. Es reicht, die Basisadresse der Struktur zu setzen und die Strukturfelder werden vom Compiler automatisch auf inkrementierte Adressen abgebildet. Es wird eine Übereinstimmung der verwendeten Datenwortbreite und Reihenfolge zur Definition in Tabelle 5.2 vorausgesetzt.

```
1 typedef struct
2 {
3   reg32_t control;
4   reg32_t status;
5   reg32_t data;
6 } volatile device_t;
```

Es wäre hilfreich, zur Laufzeit erkennen zu können, welches Gerät angesprochen wird. Dafür wurde ein Gerätedeskriptor entwickelt, welcher eine Gerätestruktur aufnehmen kann und mit einer Gerätetypkennung versieht. Die aktuell unterstützen Geräte sind:

- Zähler
- Timer
- Interrupt Controller
- GPIO Port
- UART Controller

- SPI Controller
- Display Controller
- Dateisystem
- Unbekanntes Gerät (Standard-Wert)

Zusätzlich zu den Strukturen, welche Geräte-Register abbilden, wurden allgemeine Funktionen entwickelt, die eine Arbeit mit allen kompatiblen Geräten vereinfachen. Die umgesetzten Funktionen sind in Tabelle 5.13 aufgelistet.

Kernfunktion	**Beschreibung**
`kd_await_ready` `kd_await_data`	Blockierendes Warten auf Bereitschaft oder Daten

`kd_reset`
`kd_enable`
`kd_disable`
	Gerät zurücksetzen, aktivieren und deaktivieren

`kdd_read` `kdd_write`	Datenregister des Geräts auslesen und beschreiben

`kdc_read`
`kdc_test`
`kdc_write`
`kdc_set`
`kdc_unset`
	Kontrollregister des Geräts gänzlich oder über Bitmaske auslesen und beschreiben

`kds_read` `kds_write`	Statusregister des Geräts gänzlich oder über Bitmaske auslesen

Tabelle 5.13.: Allgemeine Treiberfunktionen

Die Funktionen des UART-Treibers werden durch Aufrufe der in Tabelle 5.14 aufgelisteten Funktionen umgesetzt.

Der Lese- bzw. Schreibzugriff ist auch über eine über eine UART-Datei möglich. Diese kann man über `FILE* uf = fopen("dev/uart0", /*modus*/);` öffnen.

Der Timer wird ebenfalls über einen Treiber konfiguriert. Ein Möglichkeit, die Zeiteinheit zu setzen (tick, ns, µs, ms, s) ist gegeben. Der Timer kann als selbstzurücksetzend konfiguriert werden, das heißt dass er kontinuierlich weiterzählt und bei jedem Erreichen des Schwellwertes einen Interrupt setzt und wieder beim Initialwert beginnt. Die Treiberfunktionen sind in Tabelle 5.15 aufgelistet.

Funktion	Beschreibung	Kernfunktion
uart_reset	UART zurücksetzen	kd_reset
uart_enable	UART aktivieren	kd_enable
uart_disable	UART deaktivieren	kd_disable
uart_read	UART-Datenregister auslesen	kdd_read
uart_write	UART-Datenregister beschreiben	kdd_write

Tabelle 5.14.: Treiberfunktionen des Display-Controllers

Funktion	Beschreibung
timer_set_reload	Neuen Zählwert setzen
timer_set_unit	Zeiteinheit setzen
timer_set_autoreset	Timer automatisch zurücksetzen

Tabelle 5.15.: Treiberfunktionen des Timers

Mithilfe des Zählertreibers wird die Konfiguration vereinfacht. Es werden Strukturen und Funktionen eingeführt, um einen neuen Schwellwert zu setzen, den Zähler zurückzusetzen und zu aktivieren. Im Gegensatz zu anderen Geräten, dürfen hier nicht die generischen `kd_enable` und `kd_reset` Funktionen zum Aktivieren bzw. Zurücksetzen des Zählers verwendet werden, da der Schwellwert abhängig von der Zählrichtung zurückgesetzt wird und es zu undefiniertem Laufzeitverhalten kommen kann. Die Treiberfunktionen sind in Tabelle 5.16 aufgelistet.

Funktion	Beschreibung
counter_set_reload	Neuen Zählwert setzen
counter_reset	Zähler zurücksetzen
counter_enable	Zähler aktivieren

Tabelle 5.16.: Treiberfunktionen des Zählers

Für den Display-Controller werden in Tabelle 5.17 aufgelistete Funktionen umgesetzt. Der Zugriff auf den Videopuffer erfolgt über die Dateisystem-Abstraktionsschicht (empfohlen) über `FILE* df = fopen("dev/display0", /*modus*/);` oder alternativ direkt über die Display-Struktur bzw. generische Treiberfunktionen.

Die umgesetzten Treiberfunktionen des Interrupt-Controllers sind in Tabelle 5.18 aufgelistet. Die IRQs, die verwendet werden dürfen, entsprechen exakt den Angaben aus Tabelle 5.4.

Funktion	Beschreibung	Kernfunktion
`display_reset`	Display zurücksetzen	`kd_reset`
`display_enable`	Display aktivieren	`kd_enable`
`display_disable`	Display deaktivieren	`kd_disable`
`display_set_textmode`	Textmodus setzen	
`display_set_graphicmode`	Rastermodus setzen	
`display_flush`	Videopuffer darstellen	
`display_clear`	Displayinhalt leeren	

Tabelle 5.17.: Treiberfunktionen des Display-Controllers

Funktion	Beschreibung
`irc_is_set`	Prüfen, ob IRQ gesetzt
`irc_clear`	IRQ zurücksetzen
`irc_set_mask`	Maske für IRQ setzen
`irc_set_pol`	Polarität für IRQ setzen
`irc_set_edge`	Flankenerkennung für IRQ setzen

Tabelle 5.18.: Treiberfunktionen des Interrupt-Controllers

5.2.2. Dynamisches Speichermanagement

Das dynamische Speichermanagement ist notwendig, um zur Laufzeit neue Speicherblöcke allozieren und Programme flexibler gestalten zu können. Da für den Plasma kein vollwertiges Betriebssystem verfügbar ist und aus Speichermangel auf die Verwendung der C-Standardbibliothek verzichtet wird, ist eine eigene Implementierung der `malloc` unf `free` Funktionen notwendig. Bei einem vollwertigen Betriebssystem würden diese über einen Syscall den Kernel anweisen, einen freien Speicherblock zurückzugeben. Auch darauf wird hier verzichtet, stattdessen rufen diese unmittelbar die `kmalloc` und `kfree` Umsetzungen der Firmware auf. Die Umsetzung, die von EmbeddedArtistry [17] übernommen wird, baut auf einer verketteten Liste auf und gibt den jeweils ersten der angeforderten Größe entsprechenden Speicherblock zurück.

5.2.3. I/O

Es wird eine neue Abstraktionsschicht zur Kommunikation mit den Geräten eingeführt. Diese bietet, ähnlich wie bei UNIX-artigen Betriebssystemen, dateibasierten Zugriff auf Geräte. Beispielsweise schreiben `fprint(display_file, "TEST")` und

`fprint(uart_file, "TEST")` jeweils die Zeichenfolge „TEST“ in den Video- und UART-Puffer. Um dies zu ermöglichen, wird eine `FILE`-Datenstruktur entworfen, welche einen Gerätedeskriptor sowie Ein- und Ausgabepuffer und die aktuelle Lese/Schreibposition abstrahiert. Über Funktionen, die mit den Datei-Funktionen aus der C-Standardbiblithek kompatibel sind, lassen sich Dateistrukturen öffnen, auslesen und beschreiben. Essentiell hierfür ist die in Abschnitt 5.2.2 beschriebene dynamische Speicherallokation, mit der zur Laufzeit neue Dateistrukturen und Puffer alloziert werden können.

5.3. Portierung der Hardware für DE1-SoC

Im ersten Schritt wird mit dem DE1-SoC SystemBuilder ein neues Projekt für Quartus erstellt. Das Tool generiert zusätzlich zur Projektdatei eine Verilog-Datei mit der Top-Level-Entity und die entsprechende Datei mit den Pin-Zuordnungen (QSF). Es werden, wie in Abb. 5.1 zu sehen ist, alle Optionen ausgewählt, damit die Pin-Zuordnung vollständig ist und bei künftigen Erweiterungen nicht angepasst werden muss.

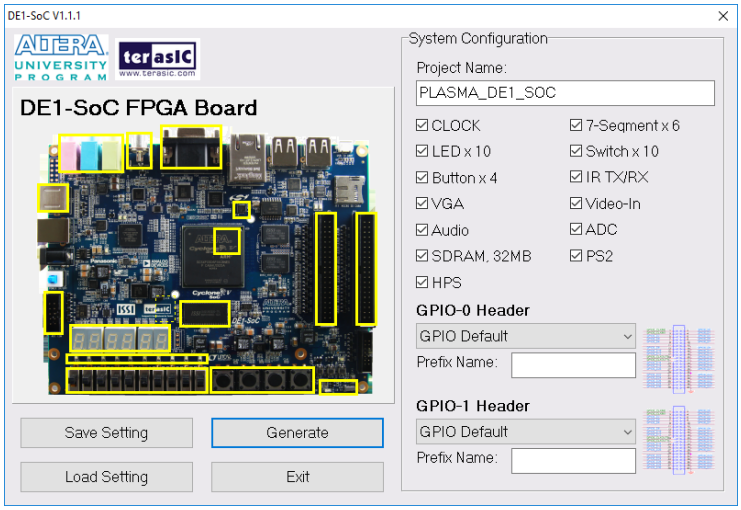


Abbildung 5.1.: DE1-SoC SystemBuilder

In der Top-Level-Entity werden die Signalzuordnungen für VGA, Audio, Video, Infrarotsender, A/D Wandler und PS2 auskommentiert, da sie nicht gebraucht werden.



Die aktuelle Revision des Entwicklungsboards hat bereits einen 64 MB SDRAM-Chip verbaut, die Angabe im SystemBuilder (32 MB SDRAM) wurde allem Anschein nach nicht aktualisiert.

Im zweiten Schritt wird das Quartus-Projekt überprüft und die tatsächliche Zielplattform zugewiesen, wie auf Abb. 5.2 dargestellt.

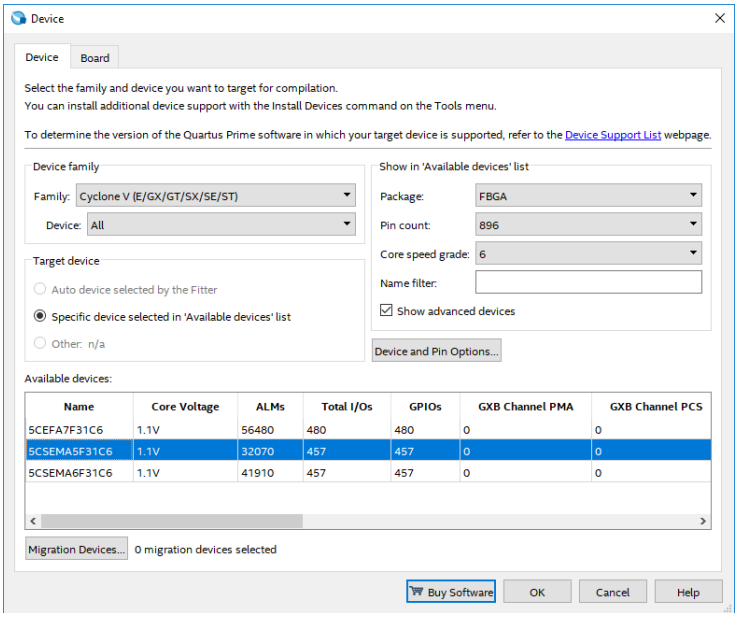


Abbildung 5.2.: Quartus Boardeinstellungen

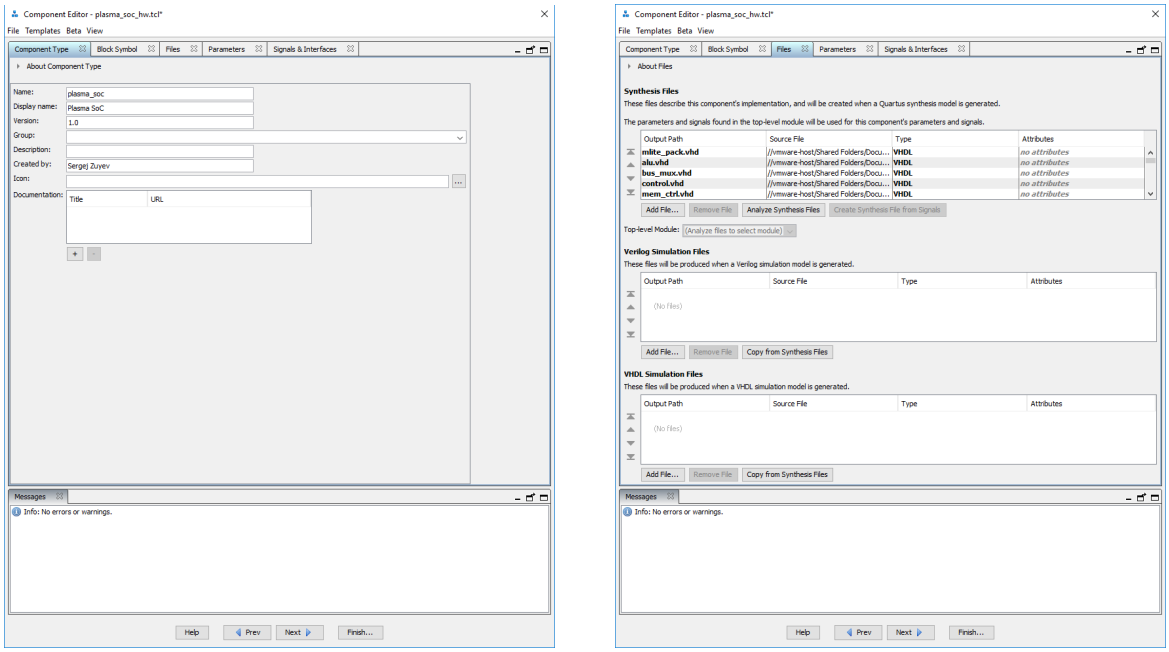
Darauffolgend wird der Platform Designer aufgerufen und über *File →New Component* ein neuer IP-Core erstellt. In Abb. 5.3 wird die Konfiguration für den Plasma-SoC IP-Core gezeigt.

Leider kann ein mit dem Platform Designer erstelltes System nicht in Modelsim simuliert werden. Die Ursache dafür ist unklar. Darum wird ein neues Projekt „quartus_no_qsys“ angelegt, welches ausschließlich das Plasma-System ohne QSYS-Komponenten beinhaltet und Simulationszwecken dient.

5.3.1. RAM

Im bestehenden System werden Xilinx-Spezifische RAM-Blöcke verwendet, dabei sind mehrere RAM-Typen für Altera bzw. Xilinx konfigurierbar. Es gilt, eine passende Variante für den Altera-FPGAs zu finden.

Dabei wird zunächst versucht, den RAM über Inferring zu implementieren. Dieses Verfahren erweist sich jedoch als ungeeignet, da bei der Synthese der gesamte im Entwicklersystem verfügbare Arbeitsspeicher inkl. Swap aufgebraucht wird:



(a) Grundeinstellungen **(b)** Quelldateien

Abbildung 5.3.: Plasma-SoC IP-Core

```
Error (114016): Out of memory in module quartus\_map.exe (24430
↪  megabytes used)

Error (114016): Out of memory in module quartus\_map.exe (44107
↪  megabytes used)
```

Als Nächstes werden die Altera Library of parameterized modules (LPM) Bausteine untersucht. Die Synthese verläuft zwar erfolgreich, jedoch wird das Beispielprogramm nicht ausgeführt. Quartus bietet mit dem In-System Memory Content Editor [18] eine Möglichkeit, den Speicherinhalt zu überprüfen und ggf. zu modifizieren. Dazu muss der LPM-RAM-Instanziierung ein zusätzliches Parameter hinzugefügt werden: *lpm_hint => „ENABLE_RUNTIME_MOD = YES, INSTANCE_NAME = <instantiation name>“* [18, S. 15–2].

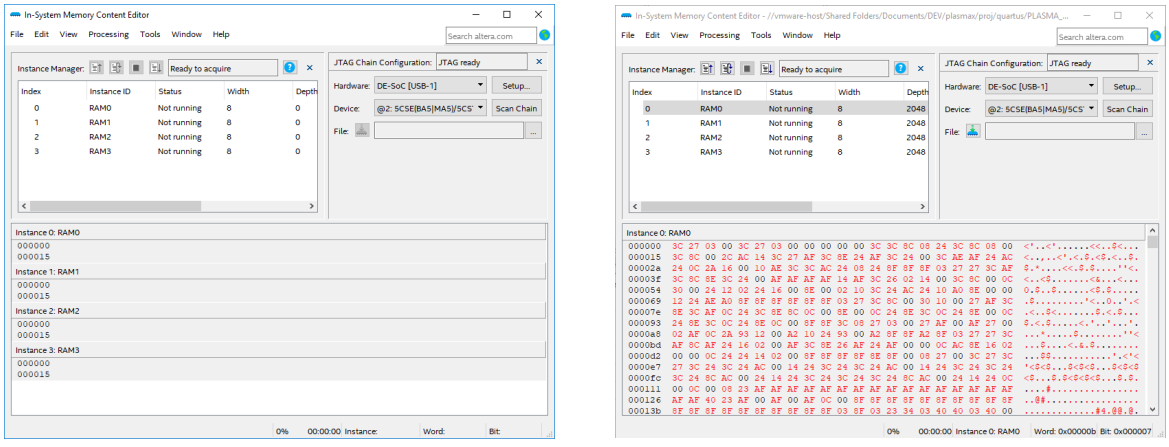
Ein weiterer wichtiger Aspekt ist die Initialisierung des Speichers. Der Speicher kann unter Anderem mit einer Memory Initialization File (MIF)-Datei (**.mif**) oder Hexadecimal (Intel-format) Datei (**.hex**) initialisiert werden [19, S. 4–18]. Der Pfad zur Initialisierungsdatei wird im Parameter *LPM_FILE* angegeben. Verwendet wird in dieser Bachelorarbeit das MIF-Format; Details werden im Abschnitt 5.5 beschrieben.

Nach einem erneuten Testlauf stürzt der Memory Content Editor beim Versuch,

den Speicherinhalt zu Laden, ab. Auffällig ist, dass die Blocktiefe 0 beträgt, wie in Abb. 5.4a zu sehen ist. Dieser Wert ist ungültig [19, S. 4–9] und muss korrigiert werden. Aus dem Synthese-Log ergibt sich der zu setzende Parameter *LPM_NUMWORDS*:

```
Info (12133): Instantiated megafunction (...) with the following
 ↪  parameter:
Info (12134): Parameter "LPM_WIDTH" = "8"
Info (12134): Parameter "LPM_WIDTHAD" = "11"
Info (12134): Parameter "LPM_NUMWORDS" = "0"
Info (12134): Parameter "LPM_INDATA" = "REGISTERED"
Info (12134): Parameter "LPM_ADDRESS_CONTROL" = "REGISTERED"
Info (12134): Parameter "LPM_OUTDATA" = "UNREGISTERED"
Info (12134): Parameter "LPM_FILE" =
 ↪  "../../build/sw/examples/sample_ram_image.vhd_block_1.mif"
Info (12134): Parameter "LPM_TYPE" = "LPM_RAM_DQ"
Info (12134): Parameter "USE_EAB" = "ON"
Info (12134): Parameter "INTENDED_DEVICE_FAMILY" = "UNUSED"
Info (12134): Parameter "LPM_HINT" = "ENABLE_RUNTIME_MOD = YES,
 ↪  INSTANCE_NAME = RAM1"
```

Nach der Parametrisierung des Speichers ist der Inhalt vorhanden und auslesbar, wie in Abb. 5.4b dargestellt.

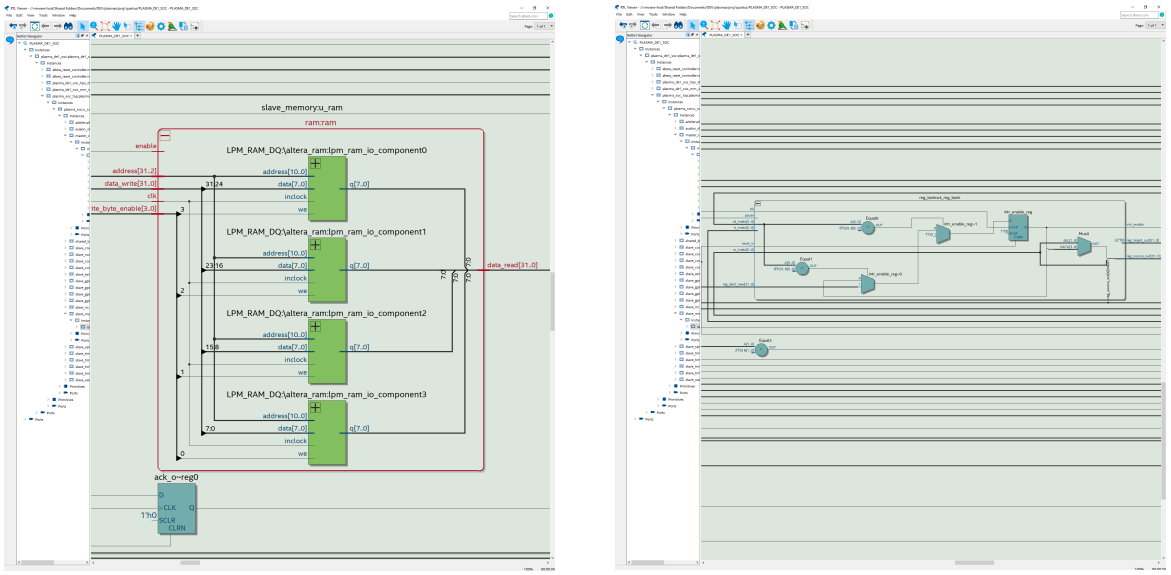


(a) Fehlerhafte LPM-Parametrisierung **(b)** Korrekte LPM-Parametrisierung

Abbildung 5.4.: Ausgelesener Speicherinhalt

5.3.2. Register-File

Das Beispielprogramm wird dennoch nicht ausgeführt. Die Vermutung, dass das als RAM-Block implementierte Register-File nicht synthetisiert wird, bestätigt sich durch eine Überprüfung der generierten Netzliste im RTL-Viewer. In Abb. 5.5 sind die Netzlisten des Arbeitsspeichers und der Register-Datei gegenübergestellt. Während in Abb. 5.5a die LPM-RAM-Elemente deutlich zu sehen sind, ist im Register-File auf Abb. 5.5b nur Steuerlogik ohne RAM-Blöcke erkennbar.



(a) RAM **(b)** REG_BANK

Abbildung 5.5.: Vergleich der RTL-Netzlisten

Es stellt sich heraus, dass für die Register-Datei der Speichertyp-Parameter mit dem ungeeigneten Wert „XILINX_16X“ konfiguriert ist und die Register-Datei deswegen nicht synthetisiert wird. Altera LPM-RAM-Blöcke erweisen sich ebenfalls als ungeeignet, da sie mit dem verwendeten Cyclone V FPGA inkompatibel sind:

```
Warning (272007): DUAL PORT RAM for Cyclone V not possible with the
 ↪  altsyncram megafunction. Implementing using benchmarking mode,
 ↪  trying to tie up the inputs using available clocks. Output
 ↪  behavior will not be the same

Error (287078): Assertion error: Can't convert dual-port RAM for
 ↪  Cyclone V device family using altsyncram megafunction because
 ↪  Cyclone V supports only synchronous dual-port RAM
```

Es wird ein durch Inferring synthetisierbarer Tri-Port Speicher mit zwei Lese- und einem Schreibport für die Register-Datei ausgewählt.

5.3.3. Laufzeitverhalten

Durch Prüfung des Speicherinhalts während der Simulation fällt eine unsaktionierte Änderung des Arbeitsspeichers auf. Eine weitreichende Analyse zeigt, dass die Umsetzung des Arbeitsspeichers das vom Wishbone-Wrapper aus dem *STB*-Signal generierte *enable*-Signal nicht nutzt. Dies hat zur Folge, dass bei einem Schreibzugriff auf einen beliebigen Wishbone-Slave der auf dem Datenbus anliegende Wert in den Speicher übernommen wird. Nach der Behebung dieses Fehlers wird das Beispielprogramm korrekt ausgeführt und die im nächsten Abschnitt beschriebene Erweiterung der Hardware beginnt.

5.4. Erweiterung der Hardware für DE1-SoC

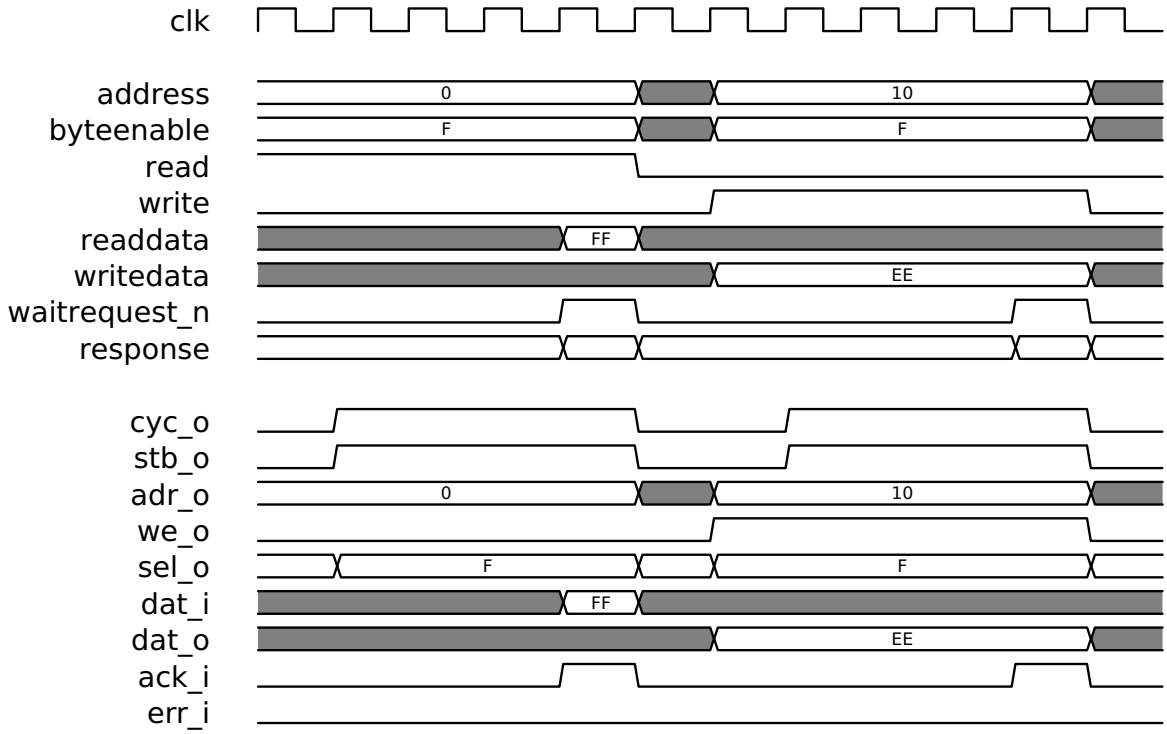
5.4.1. Avalon-Wishbone Bridge

Wie bereits im Abschnitt 3.2.3 erwähnt, nutzt ein vom Platfom Designer generiertes SoC den proprietären Avalon-Interconnect. Das Plasma-SoC hingegen setzt den Wishbone-Interconnect ein. Damit beide Systeme sinnvoll verknüpft werden können, bedarf es einer Wishbone↔Avalon-Bridge.

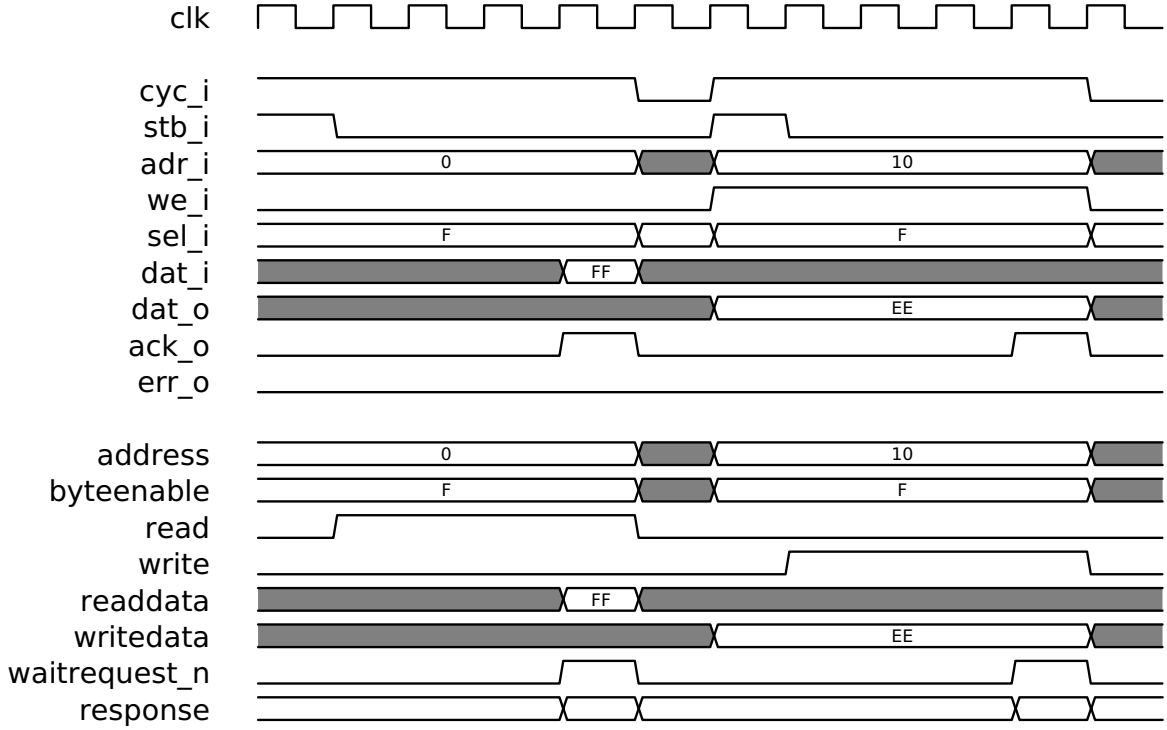
In Tabelle 5.19 ist eine Gegenüberstellung der wichtigsten Avalon-MM und Wishbone Schnittstellensignale dargestellt. Erkennbar ist, dass Takt-, Adress- und Datensignale, sowie die Steuersignale zum Großteil 1:1 zugeordnet werden können [20]. Das *chipselect*-Signal gilt als veraltet [9, Appendix A]. Darum werden die Zuordnungen zu den Steuersignalen wie *cyc_i*, *stb_i* und *we_i* durch kombinatorische Logik und Zustandsautomaten hergeleitet. Die Timingdiagramme sind jeweils für die Avalon→Wishbone- und Wishbone→Avalon-Bridge in Abb. 5.6a und Abb. 5.6b abgebildet.

5.4.2. Arbiter

Der bestehende Arbiter ist nicht funktionsfähig. Die Problematik fällt auf dem ZedBoard nicht auf, da der Wishbone-Bus dort mit nur einem Master betrieben wird. Der fehlerhafte Arbiter wird durch einen modifizierten, prioritätsbasierten Round-Robin Arbiter von Benjamin Krill [21] ersetzt.



(a) Avalon→Wishbone-Bridge



(b) Wishbone→Avalon-Bridge

Abbildung 5.6.: Wishbone↔Avalon-Bridge Timings

Wishbone Bus	**Avalon-MM**
clk_i	clk
rst_i	reset_n
cyc_i	read *or* write
stb_i	chipselect bzw. read *or* write
we_i	write *and not* read
sel_i	byteenable
adr_i	address
dat_i	readdata
dat_o	writedata
ack_o	waitrequest_n

Tabelle 5.19.: Avalon-MM und Wishbone Schnittstellen

Zunächst wird das Zweier-Komplement aus dem Request-Eingangssignal gebildet. Das Ergebnis und das Request-Eingangssignal werden mit dem UND-Operator bitweise verknüpft, wie in Gleichung (5.1) zu sehen ist. Das Endergebnis hat dann ein least significant bit (LSB) mit der höchsten Priorität [21].

$$request_{lsb} = request_i \wedge (\overline{request_i} + 1) \tag{5.1}$$

In Gleichung (5.2) wird aus dem Request-Eingangssignal und dem gespeicherten vorherigem Grant-Signal eine Maske gebildet, welche dann nach Anwendung der Gleichung (5.3) das neue Grant-Signal ergibt [21].

$$mask = request_i \wedge (\overline{grant_{prev} - 1) \vee grant_{prev}}) \tag{5.2}$$

$$grant_{next} = mask \wedge (\overline{mask} + 1) \tag{5.3}$$

Anschließend wird in Gleichung (5.4) entweder das neu generierte Grant-Signal bei einer gültigen Maske oder das Request-Signal mit der höchsten Priorität ausgewählt [21].

$$grant = \begin{cases} grant_{next}, & \text{if } mask \neq 0 \\ request_{lsb}, & \text{otherwise} \end{cases} \tag{5.4}$$

Zur Anbindung dieses Arbiters an den Wishbone-Bus wird dieser zusätzlich mit einem One-Hot dekodierten Grant-Signal versehen, da sich so der entsprechende Master mit

den Daten-, Adress- und Steuersignalen einfacher selektieren lässt. Der Wishbone-Wrapper für die CPU wird ebenfalls angepasst, sodass die Wishbone-Steuersignale *CYC* und *STB* nur bei einer gültigen Buszuteilung generiert werden.

5.4.3. Wishbone-Bus

Nach der Einführung des verbesserten Arbiters fällt auf, dass die Adresse nach Auswahl des Masters erst einen Taktzyklus später dekodiert anliegt, was zum falschen Betriebsablauf führt. Als Konsequenz wird auf eine mit dem Takt verknüpfte Auswertung und Speicherung im Register verzichtet.

Beim simulationsbasierten Testen der Avalon→Wishbone-Bridge, welche als Wishbone-Master fungiert, fällt auf, dass ein vom Slave bereitgestelltes Datenwort erst zwei Taktzyklen nach dem *ACK*-Signal anliegt. Dadurch werden fehlerhafte Werte übernommen. Um dieses Problem zu umgehen, wird aus dem Wishbone-Bus die letzte an den Takt gebundene Komponente vom Takt enkoppelt. Dadurch verursacht der Bus keine nennenswerte Verzögerung mehr und die Daten liegen zeitgleich an den Ein- und Ausgängen an. Bei höheren Taktfrequenzen könnte es jedoch zu Problemen führen.

Die Plasma-CPU erwartet die Daten jedoch immer noch mit einem Taktzyklus Versatz. Darum wird im Wishbone-Wrapper der Plasma-CPU ein Register für die Datenleitung hinzugefügt, sodass das aktuell anliegende Datenwort erst einen Takt später zur Verfügung steht.

5.4.4. Externer SDRAM

Auf dem DE1-SoC-Board ist ein SDRAM-Chip mit der Modellnummer `IS42S16320D` von ISSI Technologies verbaut. Dieser fasst $64\,\mathrm{MB}$ (32Mx16) und ist mit dem FPGA über eine $16\,\mathrm{bit}$ breite Datenleitung sowie Kontroll- und Adressleitungen verbunden [5, S. 38]. Die Grundparameter lassen sich aus Tabelle 5.20 und Tabelle 5.21 entnehmen.

Um den Speicher in Betrieb zu nehmen, ist ein SDRAM-Controller nötig. Dieser steht als IP-Core im Platform Designer zur Verfügung. Folgende Parameter werden mit neuen Werten überschrieben [23, S. 6]:

- Bits: 16
- Chip select: 1

Parameter	**-5**	**-6**	**-7**	**Einheit**
Taktzykluszeit				
$\overline{\text{CAS}}$ - Latenz = 3	5	6	7	ns
$\overline{\text{CAS}}$ - Latenz = 2	10	10	7.5	ns
Taktfrequenz				
$\overline{\text{CAS}}$ - Latenz = 3	200	167	143	MHz
$\overline{\text{CAS}}$ - Latenz = 2	100	100	133	MHz
Zugriffszeit				
$\overline{\text{CAS}}$ - Latenz = 3	5.0	5.4	5.4	ns
$\overline{\text{CAS}}$ - Latenz = 2	6	6	5.4	ns

Tabelle 5.20.: Timing Parameter [22, S. 1]

Parameter	**16M x 32**	**32M x 16**	**64M x 8**
Konfiguration	4M x 32 x 4 banks	8M x 16 x 4 banks	16M x 8 x 4 banks
Bank Adress Pins	BA0, BA1	BA0, BA1	BA0, BA1
Autoprecharge Pins	A10/AP	A10/AP	A10/AP
Zeilenadresse	512(A0 – A8)	1K(A0 – A9)	2K(A0 – A9, A11)

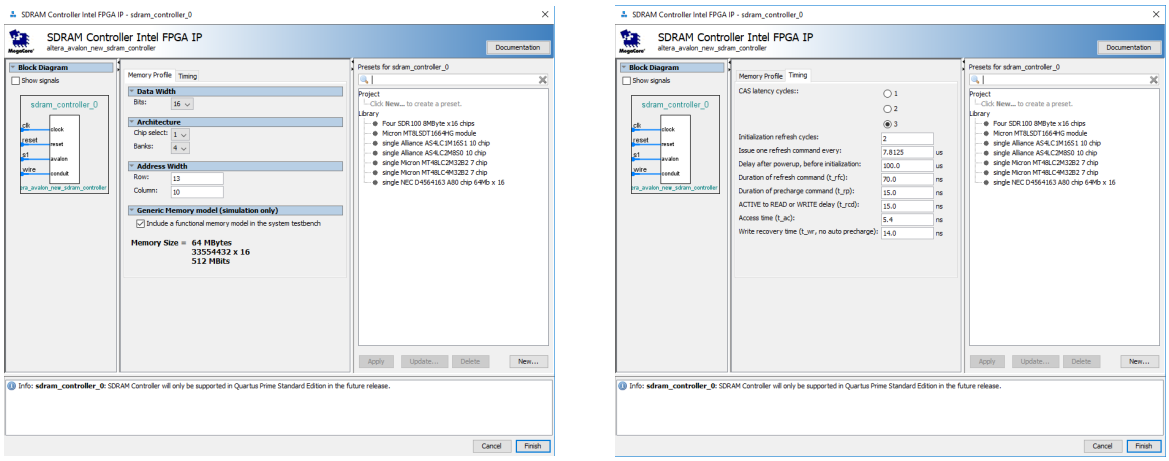
Tabelle 5.21.: Adresstabelle [22, S. 1]

- Banks: 4 (vgl. Konfiguration in Tabelle 5.21)
- Row: 13
- Column: 10 (vgl. Zeilenadresse in Tabelle 5.21)

Die Timings werden ebenfalls angepasst [23, S. 6]:

- CAS latency cycles: 3
- Issue one refresh command every: $7.8125\,\mu s$
- Duration of refresh command(t_rp): $15.0\,ns$
- Duration of precharge command(t_rcd): $15.0\,ns$
- Access time(t_ac): $5.4\,ns$ (vgl. Zugriffszeit in Tabelle 5.20)

In Abb. 5.7a und Abb. 5.7b sind die Parameter im Konfigurationsdialog des SDRAM-Controller-IP-Core abgebildet.



(a) Allgemeine Parameter **(b)** Timings

Abbildung 5.7.: Parametrisierung des SDRAM-Controllers

Die korrekte Funktion des SDRAM hängt auch von den physikalischen Charakteristika des Boards ab. Unter Anderem hat die Länge der Verbindungsleitungen zwischen dem SDRAM-Chip und dem FPGA einen Einfluss auf möglichen Taktversatz. Darum wird eine PLL benötigt, die den SDRAM-Takt dem Systemtakt unter Berücksichtigung der physikalischen Parameter um $3\,ns$ vorauseilen lässt [23, S. 13].

Dies kann durch Hinzufügen und geeignete Parametrisierung eines PLL-IP-Cores oder alternativ durch Hinzufügen des *Clocks Signals for DE-Series Board Peripherals*-IP-Cores geschehen. Die erste Variante setzt eine Berechnung der notwendigen Parameter oder eine aufwändige Analyse mit einem Oszilloskop voraus, sodass die Daten in einem korrekten Zeitfenster anliegen [24, 355ff].

Es wird die zweite Variante gewählt, da sie wesentlich einfacher ist und nur eine Auswahl der Zielhardware im in Abb. 5.8 dargestellten Konfigurationsdialog erfordert. Es werden der Referenztakt von $50\,\mathrm{MHz}$, der gewünschte Systemtakt von $50\,\mathrm{MHz}$ sowie die Zielhardware (das DE1-SoC-Board) eingestellt.

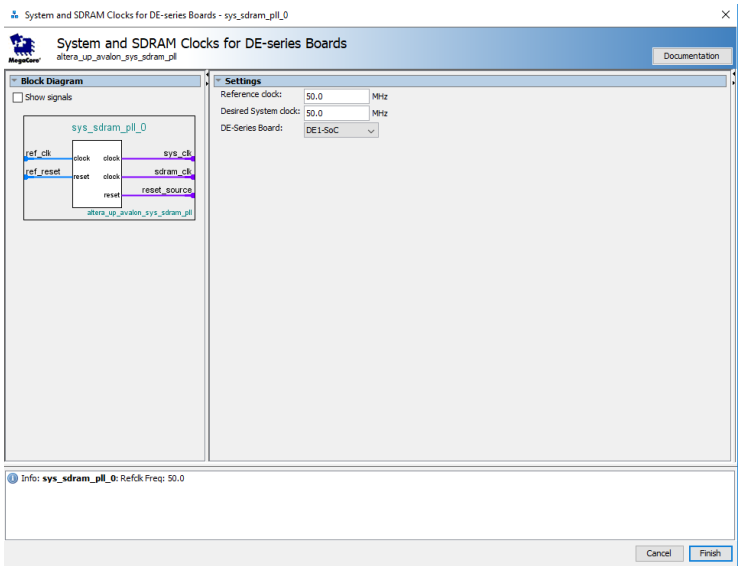


Abbildung 5.8.: Parametrisierung des *Clocks Signals for DE-Series Board Peripherals*-IP-Core

Ein wichtiger Aspekt ist, dass der verbaute SDRAM einen $16\,\mathrm{bit}$ breiten Datenbus hat. In Abb. 5.9 ist ein mit SignalTap aufgezeichneter Zugriff auf den externen SDRAM dargestellt. Von Interesse sind die blau umrahmten Wellenformen für *.../avm_read* und *.../avm_waitrequest_n*. Am oberen Rand der Abb. 5.9 ist die Zeit- bzw. Taktleiste zu sehen. Durch Abgleich der Wellenform mit der Zeitleiste lässt sich schlussfolgern, dass der gesamte Lesezyklus 6 Taktzyklen dauert. Da der SDRAM-Controller mit CAS=3 konfiguriert ist, liegen die Daten nach 3 Taktzyklen am Datenbus an. Entsprechend wird ein $32\,\mathrm{bit}$ Datenwort aus zwei $16\,\mathrm{bit}$ Transfers zusammengesetzt, welche dann zusammen 2x3 Taktzyklen ergeben.

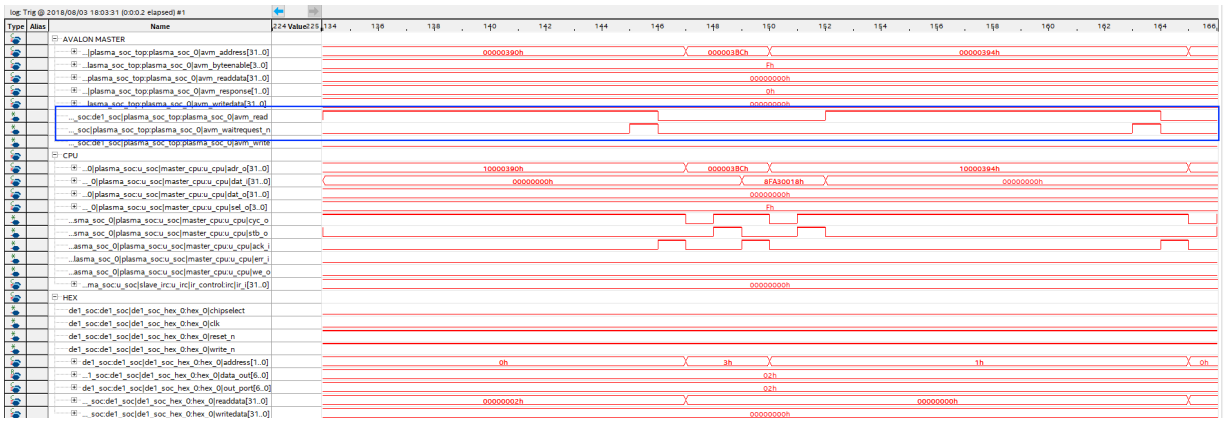


Abbildung 5.9.: Mit SignalTap aufgezeichneter Zugriff auf den externen SDRAM

5.4.5. Parallel I/O

Nach der Fertigstellung der Wishbone↔Avalon-Bridge werden im Platform Designer 8 Parallel I/O (PIO)-IP-Cores an die Bridge angebunden.

Sechs IP-Cores werden als Ausgänge mit 7 bit Datenbusbreite konfiguriert und mit sechs verfügbaren 7-Segment-Anzeigen verbunden.

Ein weiterer wird als 10 bit-Eingang konfiguriert und mit dem Schalterblock verbunden, und der letzte wird als 4 bit-Eingang mit dem Tasterblock verbunden.

5.4.6. HPS

Im Platform Designer wird ein HPS-Modul hinzugefügt. Dadurch wird die Kommunikation zwischen der ARM-CPU und dem konfiguriertem System ermöglicht.

DIe Konfiguration der FPGA-Schnittstellen ist in Abb. 5.10a dargestellt. Es werden Interrupts vom FPGA zum HPS und drei Advanced eXtensible Interface (AXI)-Bridges aktiviert:

- FPGA→HPS, 128 bit breit
- HPS→FPGA, 128 bit breit
- Lightweight HPS→FPGA, 32 bit breit

Der Avalon-Slave Port des Plasma-SoC wird an die HPS→FPGA-Bridge angebunden. Dadurch wird dem HPS der Zugriff auf den gesamten Adressraum des Plasma-SoC ermöglicht.

Die Konfiguration der Peripherie-Pins ist in Abb. 5.10b dargestellt.

Die SDRAM-Konfiguration wird zum bis auf den Memory-Vendor aus einer CERN-Projektdokumentation [25, S. 6–8] übernommen. Die Konfigurationspanels für **Phy**sical Layer (PHY)-, RAM-, Timing- und Board-Parameter sind in Abb. 5.11 abgebildet.

Nach der Konfiguration des HPS und des Arbeitsspeichers muss im Quartus zuerst der „Analysis & Synthesis“ ausgeführt werden. Danach muss über *Tools→TCL Scripts* das „hps_sdram_p0_assignments.tcl“-Skript ausgeführt werden. Der Skript-Dialog ist in Abb. 5.12 dargestellt.

Dieser Schritt ist notwendig, da vor der Konfiguration keine Timing-Parameter bekannt waren und so keine Pin-Zuweisung für den Arbeitsspeicher vorgenommen werden konnte [26, S. 46].

5.4.7. System ID

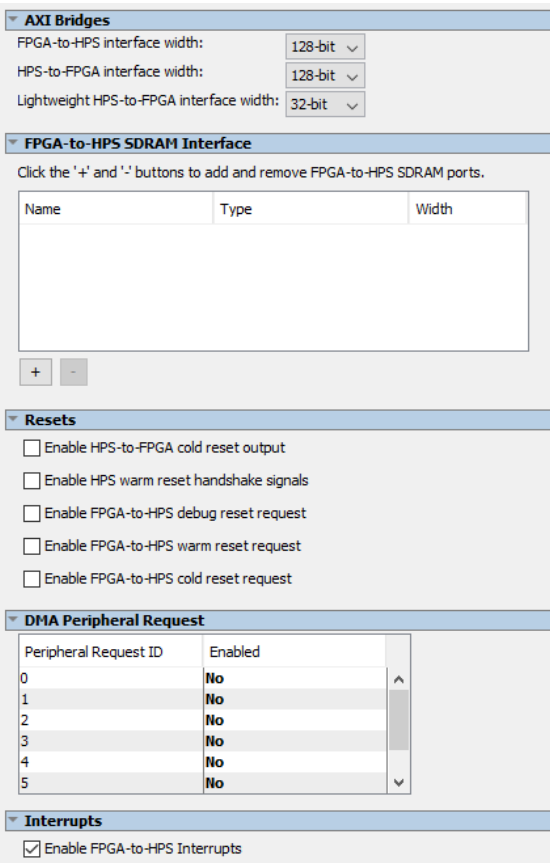
Ein System ID Peripheral IP-Core wird im Platform Designer hinzugefügt und an die Lightweight HPS→FPGA-Bridge angebunden. Für den einzigen konfigurierbaren Parameter „$32\,\mathrm{bit}$ System-ID“ wird der Wert `0xDEADBEEF` gewählt.

5.4.8. Adresszuordnung

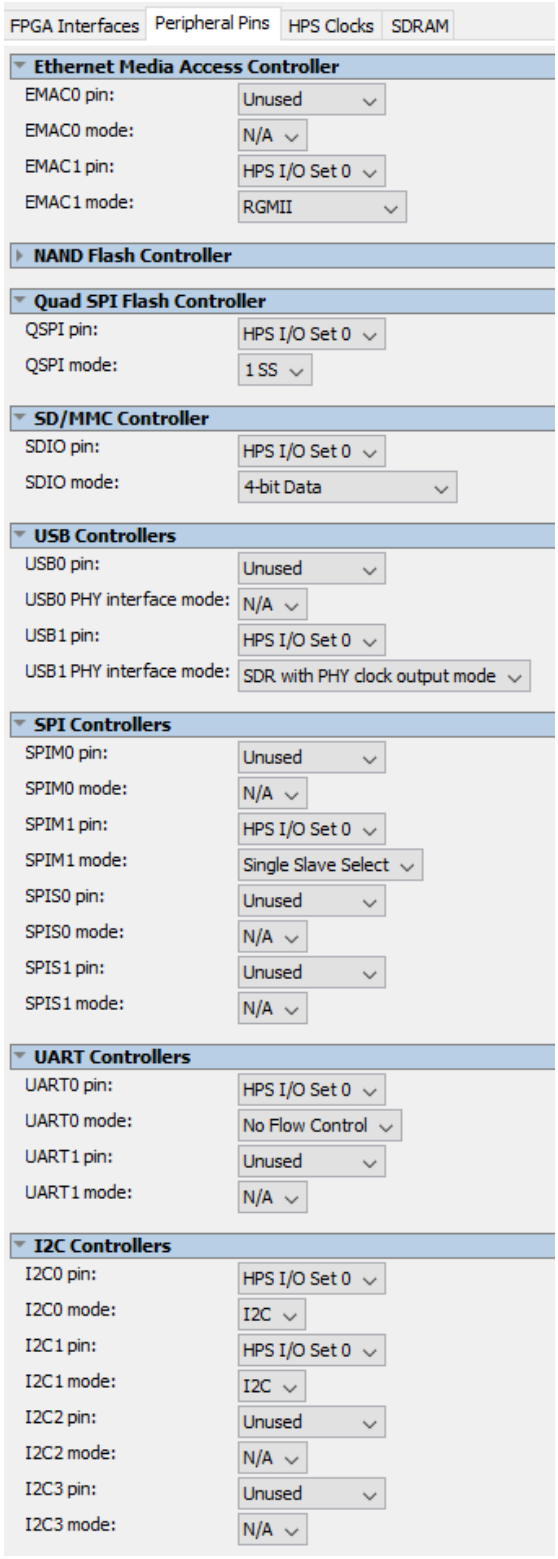
Die Adresszuordnung der Plasma-SoC-Komponenten wird überarbeitet. Dadurch ergibt sich, wie in Tabelle 5.22 aufgelistet, eine im Vergleich zur Adresszuordnung in Tabelle 5.1 und unter Berücksichtigung der neuen Komponenten verbessere Nutzung der Adressbereiche. Über die Wishbone↔Avalon-Bridge angesteuerte Slaves sind mit einem grauen Hintergrund hervorgehoben.

5.4.9. Zusammenfassung

Das Gesamtsystem besteht nun aus einem SoC-FPGA mit einem Plasma-SoC im FPGA-Bereich. Auf alle Geräte und Speicher des Plasma-SoC kann vom HPS aus zugegriffen werden, da nun, wie in Abb. 5.13 zu sehen ist, alle Komponenten über einen Bus mit dem HPS verbunden sind.

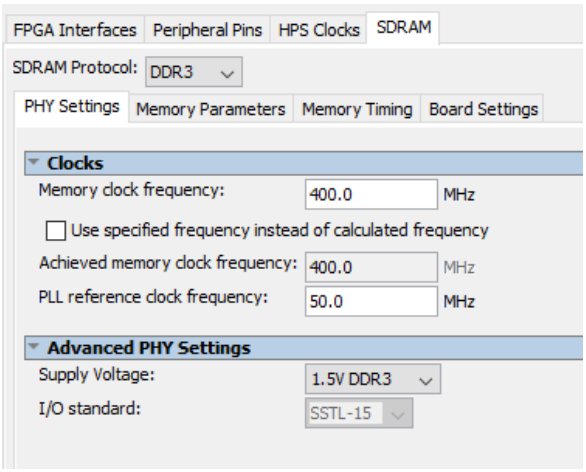


(a) FPGA-Schnittstellen

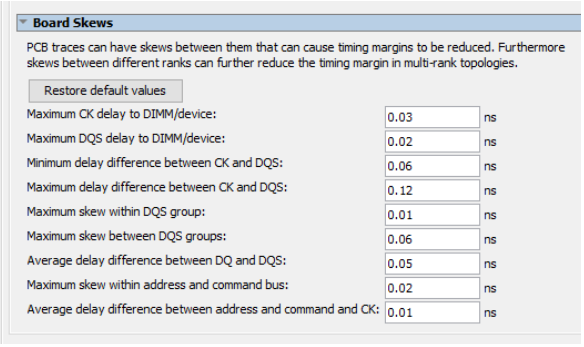


(b) Peripherie-Pins

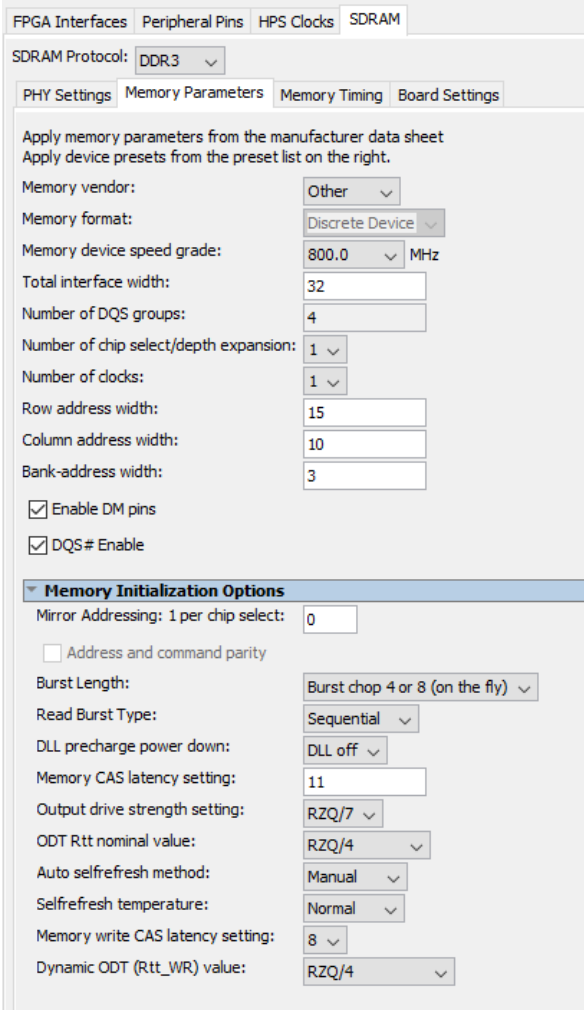
Abbildung 5.10.: HPS: Konfiguration



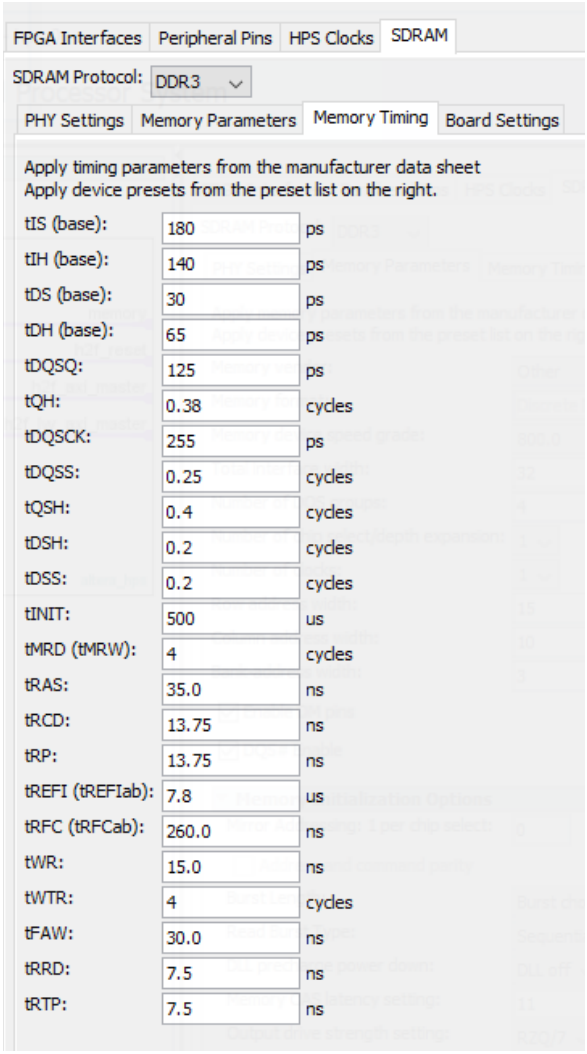
(a) PHY



(b) Board



(c) RAM



(d) Timings

Abbildung 5.11.: HPS SDRAM-Konfiguration

Gerät	Startadresse	Adressbereich
RAM_0	0x00000000	00001FFF
IRC_0	0x00005000	0x1F
$UART_0$	0x00005100	0x0F
$Timer_{0:3}$	0x00005200	0x0F
	0x00005210	0x0F
	0x00005220	0x0F
	0x00005230	0x0F
$Zähler_{0:3}$	0x00005300	0x0F
	0x00005310	0x0F
	0x00005320	0x0F
	0x00005330	0x0F
$GPIO_{0:3}$	0x00005400	0x0F
	0x00005410	0x0F
	0x00005420	0x0F
	0x00005430	0x0F
SPI_0	0x00005500	0x0F
Wishbone→Avalon-Bridge	0x10000000	0x0FFFFFFF
RAM_1	0x10000000	0x03FFFFFF
$HEX_{0:5}$	0x14000070	0x0F
	0x14000060	0x0F
	0x14000050	0x0F
	0x14000040	0x0F
	0x14000030	0x0F
	0x14000020	0x0F
$Schalter_0$	0x14000010	0x0F
$Taster_0$	0x14000000	0x0F

Tabelle 5.22.: Plasma-SoC Adresszuordnung. Grau markierte Komponenten sind an die Wishbone→Avalon-Bridge angebunden

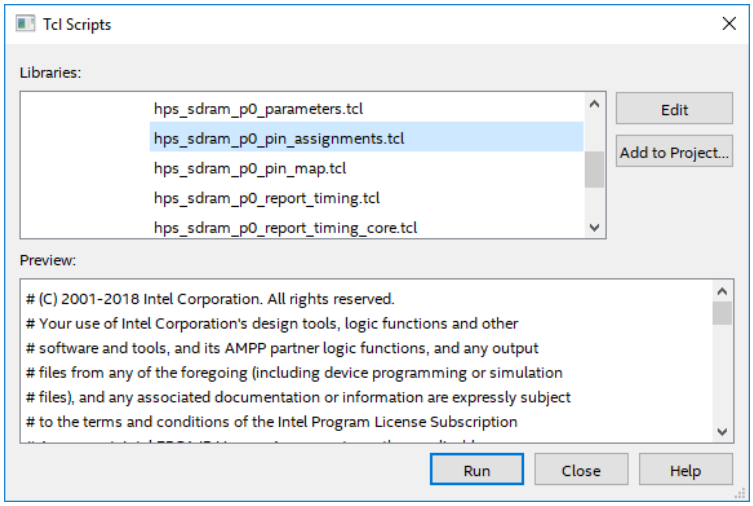


Abbildung 5.12.: Tcl Scripts: hps_sdram_p0_pin_assignments.tcl

5.5. Unterstützende Software

Zur Initialisierung des internen Arbeitsspeichers muss, wie bereits im Abschnitt 5.3 erwähnt, eine MIF-Datei verwendet werden. Um den Speicher mit einer Executable and Linkable Format (ELF)-Datei zu initialisieren, wird das `convert.py`-Skript um einen dem MIF-Format [27] entsprechenden MIF-Generator erweitert.

Ein längere Zeit bestehendes Problem des in Abschnitt 4.2 beschriebenen Systems ist das scheinbar von der Programmgröße abhängige Laufzeitverhalten. Durch eine simulationsbasierte Überprüfung des zeitlichen Verlaufs der am Wishbone-Bus anliegenden Adressen und Daten wird festgestellt, dass Variablen in den `.sdata` und `.sbss`-Sektionen überschrieben werden. Die Vermutung, dass die Linker-Konfiguration fehlerhaft ist, wird durch einen Vergleich zwischen dem standard-Linkerskript für MIPS-I-Prozessoren und der eigenen Linker-Konfiguration bestätigt.

Auffällig sind die nicht übereinstimmenden Positionen der Global Pointer (`_gp`) und die Byte-Ausrichtungen. Die eigene Linker-Konfiguration wird so korrigiert, dass der Global Pointer wie im Standard-Linkerskript zwischen den `.data` und `.sdata`-Sektionen liegt und an einer $16\,\mathrm{B}$-Grenze ausgerichtet ist. Nach dieser Korrektur werden die `.sdata` und `.sbss`-Sektionen zu keinem Zeitpunkt überschrieben und eine korrekte Ausführung der Anwendungen ist gewährleistet.

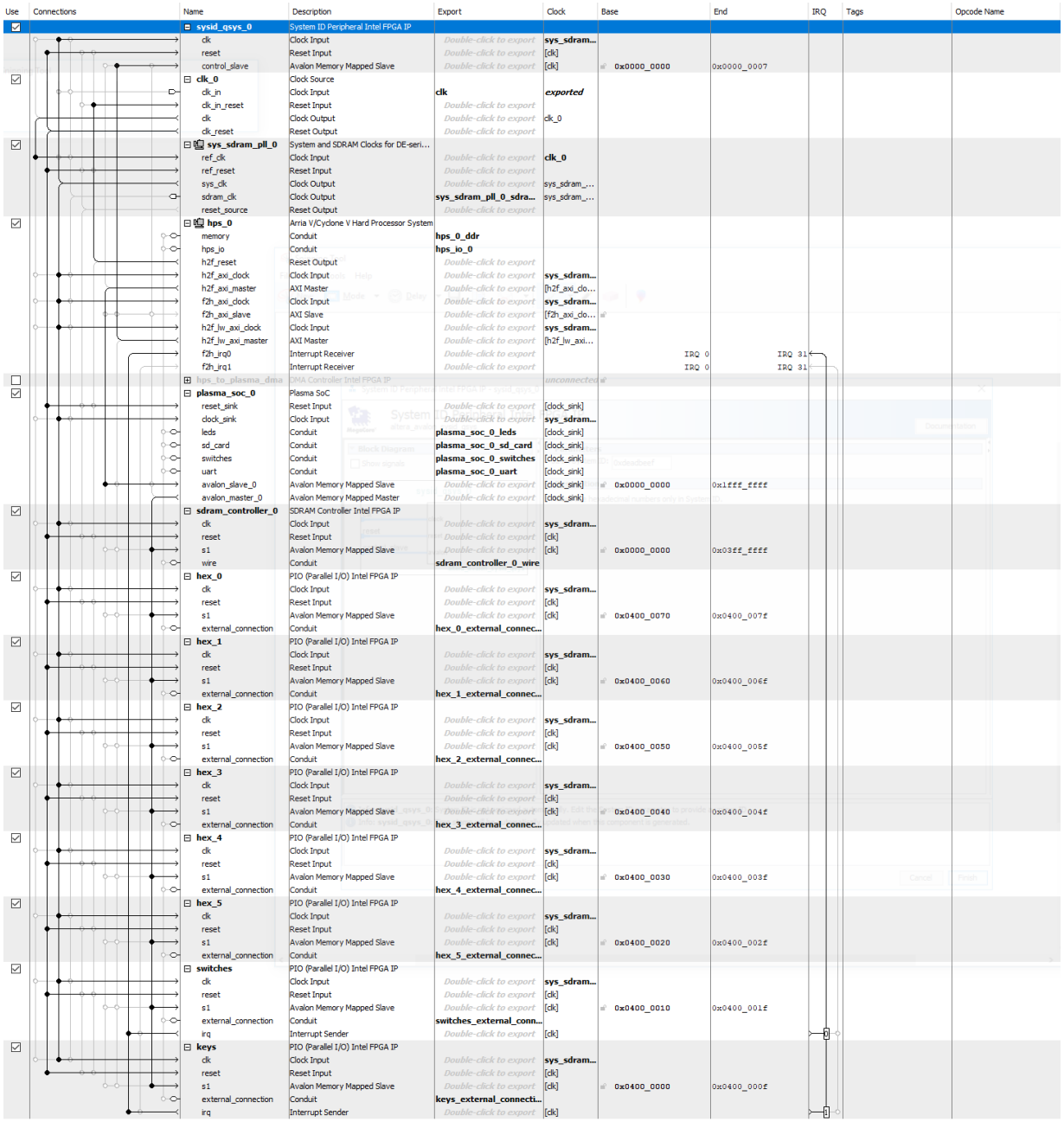


Abbildung 5.13.: Das Gesamtsystem im Platform Designer

5.6. Portierung der Plasma-Firmware und der Beispielanwendung

Die Portierung der Firmware erfolgt in Form einer neuen Systemdefinitionsdatei mit einer neuen Adresszuordnung und anderen Komponenten. Die bestehende Definition wird in die Datei `platform/plasma_soc.zedboard.h` ausgelagert, und die neue in der Datei `platform/plasma_soc.de1_soc.h` erstellt. Die CMake Build-Skripte werden um neue Einträge erweitert, welche die zwei verschiedenen Systemkonfigurationen repräsentieren. Dabei wird ein Präprozessor-Makro definiert, welcher in der Datei `plasma_soc.h` ausgewertet wird und dadurch die passende Systemdefinition einbezogen wird. Die Headerverzeichnisse der Firmware werden ebenfalls umbenannt, um Kollisionen mit den Linux-Headerverzeichnissen zu vermeiden: *dev* und *sys* werden jeweils mit dem Präfix „k“ versehen.

Aus der Beispielanwendung werden die Funktionen für die UART-Ein- und Ausgabe sowie die Displayausgabe entfernt. Stattdessen erfolgt die Anzeige der seit dem Systemstart vergangenen Zeit über die auf dem Board vorhandenen 7-Segment Anzeigen. Dafür wird eine Funktion entwickelt, welche für Ganzzahlen im Bereich von `0x0...0xF` für Common-Anode 7-Segment-Anzeigen geeignete Codes zurückgibt.

Die LED-Lauflichtfunktion wird erweitert, sodass die Richtung und Muster konfigurierbar sind. Das LED-Muster ist im externen SDRAM über die Adresse `0x800` und die Richtung über der Adresse `0x804` ansprechbar.

5.7. Einrichtung eines Linux-Betriebssystems

In diesem Abschnitt wird in mehreren Schritten vom Devicetree über Bootloader und Kernel bis hin zur Entwicklung eines Treibers ein individuell eingerichtetes Linux-Betriebssystem erstellt. Sehr hilfreich für das Verständnis ist die Embedded-Linux Anleitung von RocketBoards.org [28].

5.7.1. Devicetree

Damit im Betrieb unter Linux die Hardware richtig erkannt wird und auch eigene Treiber geladen werden können, muss eine Devicetree-Definition erstellt werden.

Zwar ist es möglich, mit der Altera SoC EDS Entwicklungsumgebung eine Device-Tree-Datei zu generieren, jedoch ist das Ergebnis nicht zufriedenstellend. Bis auf

die Plasma-SoC- und SysID-Einträge fehlen alle weiteren im Platform Designer definierten Komponenten. Außerdem führen Kompatibilitätsprobleme mit verschiedenen Kernel-Versionen zur Fehlfunktion der Treiber und/oder Absturz des Systems.

Aus diesem Grund wird gemäß Spezifikation Devicetree v0.2 [10] eine für das Plasma-SoC und das DE1-SoC-Board geeignete Devicetree-Datei `socfpga_cyclone5_de1_soc.dts` entworfen. Mit dem C-Präprozessor werden die Definitionen aus den Originaldateien `socfpga.dtsi` und `socfpga_cyclone5.dtsi` miteinander verknüpft [29, Version 4.9.78-ltsi, `arch/arm/boot/dts/`].

Nachfolgend werden die wichtigsten Fragmente beschrieben.

Die Definition des Arbeitsspeichers wird um Einträge für den internen und externen (siehe Abschnitt 5.4.4) RAM des Plasma-SoC erweitert:

```
memory@0 {
        name = "memory";
        device_type = "memory";
        reg =   <0x00000000 0x40000000>, /* 1GB DDR3 */
                <0xC0000000 0x00002000>, /* 8KB Plasma RAM0 */
                <0xD0000000 0x00010000>; /* 64MB Plasma RAM1 */
};
```

Listing 5.1: Devicetree: Arbeitsspeicherdefinition

Die HPS-Bridge-Definition wird vervollständigt. Das Wichtigste hier ist die Adress-Translation über den `ranges`-Ausdruck: Die Basisadresse `0xFF200000` (lightweight HPS-to-FPGA bridge [26, S. 21]) mit dem Adressbereich `0x8` wird auf `0,0`, die Basisadresse `0xC0000000` (HPS-to-FPGA Bridge [26, S. 22]) mit dem Adressbereich `0x5510` wird auf `1,0`, und die Basisadresse `0xD4000000` mit dem Adressbereich `0x80` wird auf `2,0` abgebildet. Eingehende Basisadressen und Adressbereiche sind im Listing 5.2 orange markiert, und die Zielbereiche entsprechend grün.

```
hps_0_bridges: bridge@c0000000 {
        compatible = "altr,bridge-1.0", "simple-bus";
        reg = < 0xC0000000 0x20000000 0xFF200000 0x00200000 >;
        reg-names = "axi_h2f", "axi_h2f_lw";
        #address-cells = < 2 >;
        #size-cells = < 1 >;
        ranges = <0 0  0xFF200000 0x00000008 >,
                 <1 0  0xC0000000 0x00005510 >,
                 <2 0  0xD4000000 0x00000080 >;
         ...
```

Listing 5.2: Devicetree: Adresstranslation in der HPS-Bridge

Dem aus der Basisadresse `0xFF200000` transliertem Adressbereich `0,0` wird im Listing 5.3 die Beschreibung des SysID-IP-Core zugeordnet.

```
hps_0_bridges: bridge@c0000000 {
...
        sysid_qsys_0: sysid@0,0 {
                compatible = "ALTR,sysid-13.1", "ALTR,sysid-1.0",
                 ↪  "altr,sysid-1.0";
                reg = <0 0 0x00000008>;
                id = < 3735928559 >;
                timestamp = < 1536342590 >;
        };
...
```

Listing 5.3: Devicetree: SysID-Definition

Weiter werden im Listing 5.4 die Beschreibungen für die im Plasma-SoC am Wishbone-Bus angebundenen Timer, Zähler und GPIOs definiert. Diese sind dem aus der Basisadresse `0xC0000000` transliertem Adressbereich `1,0` zugeordnet.

```
hps_0_bridges: bridge@c0000000 {
...
        timer@1,5200 {
                compatible = "plsoc,timer";
                reg = <1 0x5200 0x10>;
        };

... weitere 3 timer

        counter@1,5300 {
                compatible = "plsoc,counter";
                reg = <1 0x5300 0x10>;
        };

... weitere 3 Zähler

        gpio@1,5400 {
                compatible = "plsoc,gpio";
                reg = <1 0x5400 0x10>;
```

```
        };

... weitere 3 GPIOs

        spi@1,5500 {
                compatible = "plsoc,spi";
                reg = <1 0x5500 0x10>;
        };
...
```

Listing 5.4: Devicetree: Komponenten im Plasma-SoC

Zum Schluss werden die über die Avalon-Wishbone-Bridge angebundenen sechs 7-Segment-Anzeigen, der Schalterblock und der Tasterblock beschrieben:

```
hps_0_bridges: bridge@c0000000 {
...
        hex0: gpio@2,70 {
                compatible = "altr,pio-15.0", "altr,pio-1.0";
                reg = <2 0x70 0x00000010>;
                altr,gpio-bank-width = <7>;
                #gpio-cells = <1>;
                gpio-controller;
        };

... weitere fünf 7-Segment-Anzeigen

        switches: gpio@2,10 {
                compatible = "altr,pio-15.0", "altr,pio-1.0";
                reg = <2 0x10
                 ↪  0x00000010>;
                altr,gpio-bank-width = <10>;
                #gpio-cells = <1>;
                gpio-controller;
        };

        keys: gpio@2,0 {
                compatible = "altr,pio-15.0", "altr,pio-1.0";
                reg = <2 0x00 0x00000010>;
                altr,gpio-bank-width = <4>;
                #gpio-cells =
                 ↪  <1>;
```

```
                gpio-controller;
        };
...
```

Listing 5.5: Devicetree: Komponenten am Avalon-Bus

Im letzten Schritt wird der Devicetree in ein Binärformat kompiliert, damit er vom Kernel verwendet werden kann. Dafür wird der Devicetree-Compiler `dtc` benötigt.

Schwierigkeiten bereitet die Zusammenführung von drei Devicetree-Dateien mit dem C-Präprozessor zu einer, da der Devicetree-Compiler die zusammengeführte Datei nicht akzeptiert. Eine Untersuchung ergibt, dass der Devicetree-Compiler mehrere aufeinanderfolgende DTS-Header „/dts-v1/;“ nicht verarbeiten kann. Diese Wiederholungen entstehen durch die Vorverarbeitung mit dem C-Präprozessor. Ein Update des Devicetree-Compilers behebt das Problem auch nicht.

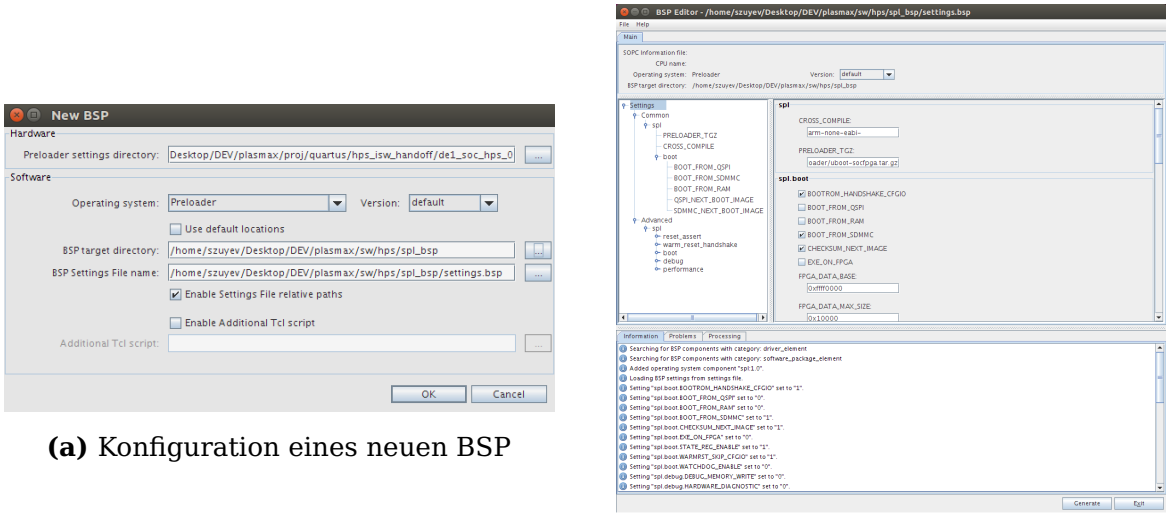
Als Workaround wird ein Shell-Skript entwickelt, welches die Devicetree-Dateien mit dem C-Präprozessor zusammenführt und mit einem regulären Ausdruck alle außer dem ersten Vorkommen von „/dts-v1/;“ entfernt. Diese Nachbearbeitung führt dazu, dass die Devicetree-Dateien erfolgreich kompiliert werden.

5.7.2. Bootloader

Für das DE1-SoC werden aus den Platform-Designer Daten der Preloader und der Bootloader „U-Boot“ generiert. Hierzu wird das Programm `bsp-editor` aus dem SoC-EDS-Paket aufgerufen und ein neues BSP erstellt. Im sich öffnendem Dialog, welcher in Abb. 5.14a dargestellt ist, wird in das Feld „Preloader settings directory“ der Pfad zum *hps_isw_handoff/de1_soc_hps_0*-Ordner im Quartus-Projektverzeichnis eingetragen. In diesem Verzeichnis befinden sich durch den Platform-Designer generierte C-Headerdateien mit Parametern zur Konfiguration der Taktgeneratoren, PLLs, RAMs, IOs und Pin-Multiplexer.

Im Hauptfenster des BSP-Editors (siehe Abb. 5.14b) werdem die Parameter `spl-.performance.SERIAL_SUPPORT` und `spl.reset_assert.FPGA_DMA[0-7]` aktiviert, und die Parameter `spl.boot.WATCHDOG_ENABLE` und `spl.warm_reset_handshake.SDRAM` deaktiviert.

Im nächsten Schritt wird der Preloader generiert. Im Ausgabeverzeichnis befindet sich der generierte Preloader und nach Ausführung des Befehls `make uboot` auch der „U-Boot“-Bootloader. In der Datei `uboot-socfpga/include/configs/socfpga_-common.h` werden die Konfigurationsvariablen `mmcloadpart` mit „2textbackslash0“



(a) Konfiguration eines neuen BSP

(b) Parametrisierung

Abbildung 5.14.: BSP-Editor

und `callscript` mit „if fatload mmc 0:$mmcloadpart $fpgadata $scriptfile“ parametrisiert.

5.7.3. Kernel

Für das DE1-SoC wird ein moderner Linux-Kernel konfiguriert und kompiliert, da die vorgefertigten Systemabbilder von Terasic und Altera alte Kernel mit Devicetree-Kompatibilitätsproblemen einsetzen.

Das „linux-socfpga“-Repository [29] wird geklont und das Release 4.9.78-ltsi ausgecheckt. Mit der Altera Embedded Command Shell [30, S. 14] wird eine Cross-Compiler Umgebung vorbereitet und der Kernel mit dem menugeführtem Konfigurationswerkzeug `nconfig` konfiguriert.

Im Konfigurator werden Parameter wie z.B der Hostname gesetzt, Gerätetreiber und Debug-Möglichkeiten ausgewählt und Altera-spezifische Einstellungen vorgenommen. Aus der Kernel-Konfiguration wird eine `defconfig`-Datei generiert, die wie eine Patch-Datei jederzeit auf einer Kernel-Konfiguration angewandt werden kann.

Kompiliert werden ein zImage und alle ausgewählten Kernel-Module.

5.7.4. Systemabbild

Ein im Altera University Program verfügbares Systemabbild[1] für das DE1-SoC-Board wird als Basis für eigene Anpassungen verwendet. In diesem Abbild ist ein Ubuntu 12.04 Betriebssystem vorkonfiguriert. Um eine auf das DE1-SoC-Board mit einem Plasma-SoC zugeschnittene Betriebssystem-Distribution mit graphischer Oberfläche zu erstellen, muss nur wenig Aufwand betrieben werden.

Es werden der Preloader, der „U-Boot“-Bootloader, das Boot-Skript, der kompilierte Devicetree sowie der Kernel mit den Kernel-Modulen durch neue Komponenten aus Abschnitt 5.7.2, Abschnitt 5.7.3 und Abschnitt 5.7.1 ersetzt.

5.8. Entwicklung von Linux-Anwendungen für das HPS

Aufbauend auf einem Beispielprogramm zur Kommunikation mit dem FPGA [30, 16ff] werden zwei mit dem Plasma-SoC kommunizierende Linux-Anwendungen entwickelt: „SysID“ und „LED Pattern Manager“.

Für einen Zugriff auf Hardware-Ressourcen müssen unter einem Betriebssystem mit virtuellem Speichermanagement physische Adressen in virtuelle abgebildet werden. Das Linux-Betriebssystem ist ein System mit virtuellem Speichermanagement und bietet die zur Adressabbildung notwendige Funktionalität an.

Mit einem Aufruf von `open(/dev/mem", (O_RDWR | O_SYNC))` wird zunächst das Memory-Device zum Lesen und Schreiben geöffnet, und mit einem Aufruf von `mmap` werden die physischen Adressräume der übergebenen Bridge-Adressen in den virtuellen Adressraum abgebildet.

In beiden Anwendungen wird eine `goto`-basierte Fehlerbehandlung umgesetzt. Dadurch werden sowohl im Fehlerfall als auch im normalen Betrieb die Ressourcen in der korrekten Reihenfolge wieder freigegeben. Außerdem wird zur Ausgabe von Fehlermeldungen die Funktion `void perror(const char *s)` verwendet. Diese Funktion gibt nicht nur die benutzerdefinierte Fehlermeldung aus, sondern auch die Beschreibung der Fehlerursache [31, 50f].

Die Anwendung „SysID“ wird zum Testen der Lightweight HPS2FPGA- und der Wishbone↔Avalon-Bridges entwickelt. Es findet ausschließlich ein Lesezugriff auf

[1] `ftp://ftp.altera.com/up/pub/Altera_Material/16.0/Linux_Images/`

die im SysID-IP-Core fest hinterlegten System-ID und Zeitstempel. Diese werden ausgelesen und im Standard-Ausgabestream ausgegeben.

Die Anwendung „LED Pattern Manager“ wird zum Testen der Lightweight HPS2FPGA, HPS2FPGA- und der Wishbone↔Avalon-Bridges entwickelt. Es finden sowohl Lese- als auch Schreibzugriffe statt.

Analog zur „SysID“-Anwendung werden die System-ID und der Zeitstempel aus dem SysID-IP-Core ausgelesen und im Standard-Ausgabestream ausgegeben.

Zusätzlich werden jede $500\,\mathrm{ms}$ die Werte des Schalter- und Tasterblocks über die HPS2FPGA-Bridge ausgelesen und ausgewertet. Der Wert des Schalterblocks wird in den externen RAM des Plasma-SoC geschrieben, und vom dort ausgeführtem Programm als LED-Muster angezeigt. Der Tasterblock ist active-low: im Normalzustand liegt eine logische „1“ an, und bei Aktivierung liegt eine logische „0“ an. Durch bitweise Negation und Maskierung der unteren 4 Bit wird ein active-high Wert ermittelt.

Wird *KEY3* betätigt, so ändert sich die Bewegungsrichtung des LED-Musters zur Linksbewegung, *KEY2* hingegen bewirkt eine Rechtsbewegung. Das Drücken des *KEY1*-Tasters halbiert das Timer-Intervall, sodass die Bewegungsgeschwindigkeit verdoppelt wird. Das Betätigen des *KEY0* verlangsamt die Bewegung durch Verdoppelung des Timer-Intervalls.

Da die Schalter und Taster in einer Endlosschleife abgefragt werden und keine Konsoleninteraktion genutzt wird, kann die Anwendung nicht ordentlich beendet werden. Damit die Ressourcen trotzdem freigegeben werden, wird ein benutzerdefinierter Signal-Handler für das Interrupt-Signal (`SIGINT`) eingeführt. Dafür wird die Funktion `signal(int signum, void *handler())` verwendet [31, 88f].

5.9. Entwicklung eines Linux-Treibers für das HPS

Um eine gute Integration des Plasma-SoC und dem Linux-Betriebssystem auf dem HPS zu demonstrieren, wird ein einfacher Treiber für über Wishbone angebundene Zähler im Plasma-SoC entwickelt. Mit diesem Treiber wird der Zugriff auf das Reload-Register des Zählers über sysfs ermöglicht. Als Vorlage dient der Altera SysID-Treiber [29, Version 4.9.78-ltsi, `drivers/misc/altera_sysid.c`].

Im Abschnitt 5.7.1 wurde bereits eine Devicetree-Datei für den Plasma-SoC auf dem DE1-SoC-Board entworfen. Für diesen Abschnitt sind die im Listing 5.4 dargestellten Definitionen sämtlicher am Wishbone-Bus angebundener Komponenten des Plasma-SoC von hoher Bedeutung. Über den `compatible`-Eintrag wird der Treiber an alle

im Devicetree definierten Geräte mit übereinstimmenden `compatible`-Schlüsselwert gebunden, in diesem Fall an die Zähler.

Bei der Registrierung des Treibers wird ein sysfs-Verzeichnis im übergeordneten Bus-Verzeichnisses erstellt. In diesem Verzeichnis kann der Treiber eine Schnittstelle exportieren, sodass eine Ansteuerung aus dem Userspace möglich wird [29, Version 4.9.78-ltsi, `Documentation/driver-model/driver.txt, Zeile 131`].

Um eine Schnittstelle im sysfs zu exportieren, müssen Attribute und dazugehörige Getter- und Settter (in der Kernel-Terminologie: `show` und `store`) implementiert werden [29, Version 4.9.78-ltsi, `Documentation/filesystems/sysfs.txt`, Zeile 233].

Sobald eine Bindung zu einem Zähler im Plasma-SoC möglich wird, findet automatisch ein Aufruf der „plsoc_counter_probe“-Methode statt. Der Adressraum des Zählers wird mit `platform_get_resource` abgerufen und mit `devm_ioremap_resource` auf den mit `devm_kzalloc` allozierten und initialisierten Speicher abgebildet. Abschließend werden die Attribute mit `sysfs_create_group` zum sysfs hinzugefügt, sodass ein Zugriff auf den Zähler über `/sys/devices/platform/...` möglich wird.

Die „plsoc_counter_remove“-Methode wird beim Entladen des Treibers bzw. Deaktivierung des Zählers aufgerufen. Es werden die sysfs-Einträge entfernt und der Adressraum des Zählers vom Treiber entbunden.

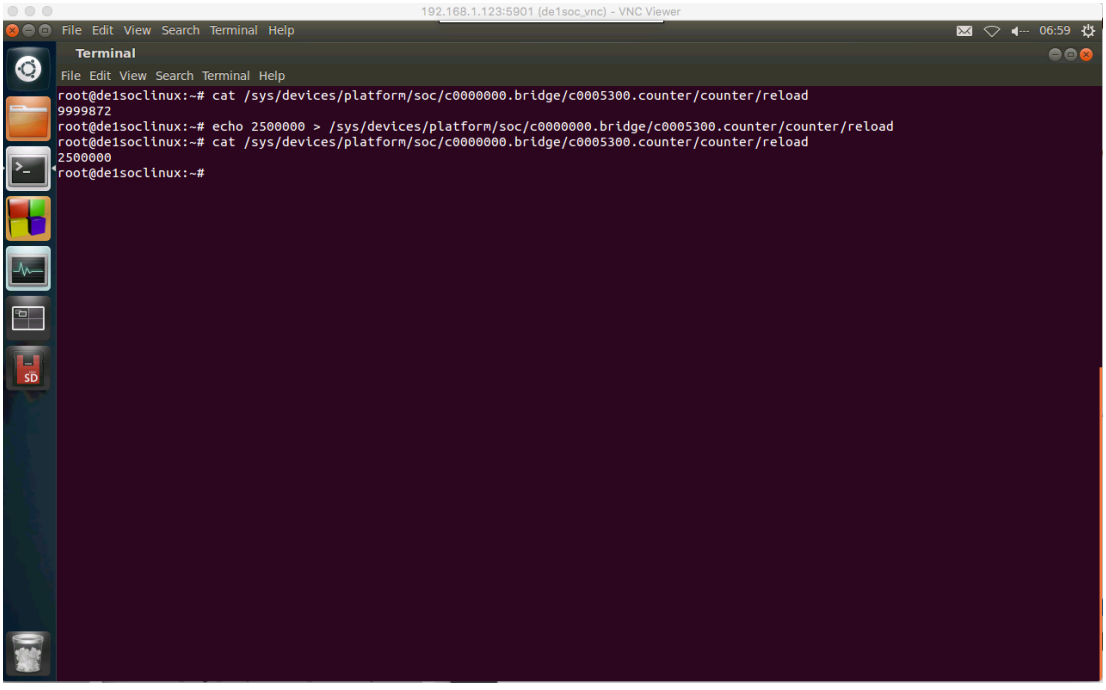


Abbildung 5.15.: Zugriff auf das Reload-Register aus dem Terminal. Es wird der erste Zähler mit der Wishbone-Adresse `0x5300` angesteuert.

6. Ergebnisse und Ausblick

In diesem Kapitel werden die Ergebnisse dieser Bachelorarbeit zusammengefasst und künftige Entwicklungsmöglichkeiten erörtert. Es werden sowohl neue Möglichkeiten, als auch Einschränkungen des entstandenen Systems besprochen.

6.1. Zusammenfassung

Mit der Einführung des Wishbone-Bus wird ein Ausbau des Plasma-Systems mit eigens entwickelten Komponenten und frei verfügbaren IP-Cores von OpenCores deutlich vereinfacht, da vorher keine standardisierte Schnittstelle zur Plasma-CPU gegeben war.

Die Erweiterung des Plasma-Systems um eine Wishbone↔Avalon-Bridge ermöglicht die Integration als IP-Core in Altera/Intel-FPGA-basierte Projekte. Erst mit Einführung dieser Bridge wird eine einfache Anbindung der DE1-SoC-spezifischen Komponenten, wie dem verlötetem SDRAM-Chip, der ARM-CPU, Schalter, Taster, LEDs und 7-Segmentanzeigen, an das Plasma-SoC möglich.

Durch die Anbindung des externen SDRAM-Chips erhöht sich der im Plasma-System verfügbare Arbeitsspeicher von $8\,\mathrm{kB}$ auf über $64\,\mathrm{MB}$, was die Verarbeitung größerer Datenmengen und komplexere Anwendungen ermöglicht.

Korrekturen des Wishbone-Busses und die Einführung eines neuen Bus-Arbiters legen den Grundstein für einen ordnungsgemäßen Multi-Master-Betrieb. Das System arbeitet in diesem Modus sobald ein Zugriff auf das Plasma-SoC über die Wishbone↔Avalon-Bridge erfolgt.

Die Erstellung eines Linux-Betriebssystems auf Basis eines vorgefertigten Systemabbilds ist ein wichtiger Schritt zur vollen Nutzung der Möglichkeiten, die ein SoC-FPGA bietet. Das Betriebssystem wird auf dem ARM-Prozessor ausgeführt, und greift über einen Treiber und mehrere Benutzeranwendungen auf das Plasma-SoC zu. Um den Zugriff auf das Plasma-SoC zu ermöglichen, wurde ein aktueller Linux-Kernel konfiguriert und kompiliert, die Linux-Treiber aktualisiert und eine eigene,

auf das DE1-SoC-Board und Plasma-SoC zugeschnittene Devicetree-Beschreibung entwickelt.

Der entwickelte Treiber steuert die Reload-Register der Zähler im Plasma-SoC an.

Leider hat die Bearbeitungszeit nicht mehr ausgereicht, um das Plasma-SoC mit einer ausführbaren Datei aus dem Linux-Betriebssystem heraus zu programmieren. Aus dem selben Grund konnten für das Plasma-SoC keine Regressionstests auf dem Digilent Zedboard durchgeführt werden.

6.2. Ausblick

Der Wishbone-Bus ist in der einfachsten Variante umgesetzt und unterstützt keine Bursttransfers, atomare Read-modify-write (RMW)-Zyklen und split transactions. Pipelining wurde ebenfalls nicht erprobt, da dafür eine Erweiterung der Wishbone-Slaves notwendig wäre. Für einen performanteren Betriebsablauf könnten diese Features implementiert werden.

Gegebenenfalls könnte eine Plasma-SoC CSR-Komponente entwickelt werden, die das Auslesen von Statusflags und Anhalten des Plasma-Prozessors von außen ermöglicht. Ein denkbarer Anwendungsfall wäre die Umprogrammierung, da das Verhalten der Plasma-CPU beim Überschreiben des Programmspeichers im laufenden Betrieb derzeit nicht absehbar ist.

Zudem könnten Linux-Treiber für weitere Komponenten des Plasma-SoC, wie z.B. Timer und GPIOs, sowie ein Treiber, welcher die Speicher des Plasma-SoC als eine konsekutive Datei abbildet, entwickelt werden.

Basierend auf dem ELF-Converter-Skript könnte für den Plasma-Prozessor ein ELF-Loader entwickelt werden, sodass eine Programmierung aus dem Linux-Betriebssystem heraus möglich wird.

Da dem Plasma-SoC nun deutlich mehr Arbeitsspeicher zur Verfügung steht, wäre die Portierung eines Embedded-Betriebssystems wie z.B FreeRTOS[1] oder Zephyr[2] eine erwägenswerte Verbesserungsmöglichkeit.

[1] `https://www.freertos.org/`
[2] `https://www.zephyrproject.org/`

Literaturverzeichnis

[1] S. Rhoads. (21. Nov. 2016). Plasma - most MIPS I(TM) opcodes, Adresse: https://opencores.org/project/plasma (besucht am 22. 04. 2018).

[2] B. Zeller, „SoC-based Image Classification using Binarized Neural Networks“, Aug. 2018.

[3] *ZedBoard, (Zynq™Evaluation and Development) Hardware User's Guide*, Version 2.2, AVNET, 27. Jan. 2014.

[4] *Architecture Brief, What is an SoC FPGA?*, Altera, 2014.

[5] *DE1-SoC, User Manual*, V2.0.3, Terasic, 24. Aug. 2016.

[6] P. P. Chu, „Embedded SoPC design with Nios II Processor and VHDL examples“, in. John Wiley & Sons, Inc, 2011, ISBN: 978-1-118-00888-1.

[7] M. Mitic, M. Stojcev und Z. Stamenkovic, „An Overview of SoC Buses“, 11. Juli 2007.

[8] W. D. P. et al. (2010). Wishbone B4, WISHBONE System-on-Chip (SoC) Interconnection Architecture for Portable IP Cores, OpenCores, Adresse: https://cdn.opencores.org/downloads/wbspec_b4.pdf (besucht am 02. 04. 2018).

[9] *Avalon®Interface Specifications, Updated for Intel®Quartus®Prime Design Suite: 18.0*, MNL-AVABUSREF, Intel, 22. Mai 2018.

[10] *Devicetree Specification, Release v0.2*, devicetree.org, 20. Dez. 2017.

[11] *Introduction to Device Trees*, AN5125, Freescale Semiconductor, Inc., Sep. 2015.

[12] *Altera SoC Linux Intro Workshop*, Altera, 2016.

[13] boschmitt. (9. Okt. 2014). Wishbone, Wishbone Interconnection (INTERCON), Adresse: https://github.com/boschmitt/wishbone (besucht am 02. 04. 2018).

[14] P. Tofahrn, „Interrupt Controller, für die Plasma-CPU“, 30. Sep. 2017.

[15] R. Kim und M. Mattioli. (7. Mai 2017). ZedBoard OLED Display, Adresse: https://github.com/mmattioli/ZedBoard-OLED (besucht am 02. 04. 2018).

[16] S. et al. (25. Juli 2017). NexysVideo, Adresse: https://github.com/Digilent/NexysVideo/tree/master/Projects/GPIO/src/hdl (besucht am 02. 04. 2018).

[17] P. Johnston. (15. Feb. 2017). Implementing Malloc, First-fit Free List, Adresse: https://embeddedartistry.com/blog/2017/2/9/implementing-malloc-first-fit-free-list (besucht am 02. 04. 2018).

[18] *In-System Modification of Memory and Constants*, QII53012-12.0.0, Altera, Juni 2013.

[19] *Internal Memory (RAM and ROM), User Guide*, UG-01068-5.0, Altera, Juni 2014.

[20] X. Xing, C. Zezong, J. Jing und K. Hengyu, „Porting from Wishbone Bus to Avalon Bus in SoC Design“, in *2007 8th International Conference on Electronic Measurement and Instruments*, (16.–18. Aug. 2007), School of Electronic Information, Wuhan University, Wuhan 430079, China, IEEE, Okt. 2007. DOI: 10.1109/ICEMI.2007.4350589.

[21] B. Krill. (11. Okt. 2016). VHDL Round-Robin Arbiter, Adresse: https://www.krll.de/portfolio/vhdl-round-robin-arbiter (besucht am 27. 06. 2018).

[22] *IS42/45R86400D/16320D/32160D IS42/45S86400D/16320D/32160D, 16Mx32, 32Mx16, 64Mx8 512Mb SDRAM*, Rev. B, Integrated Silicon Solution, Inc., 12. Mai 2015.

[23] *Using the SDRAM on Altera's DE1-SoC Board with VHDL Designs, For Quartus II 14.0*, Altera, Aug. 2014.

[24] *Embedded Peripherals IP User Guide, Updated for Intel® Quartus® Prime Design Suite: 18.0*, UG-01085, Intel, 7. Mai 2018.

[25] A. Z. Rasheed, „Project Documentation, Configuration of SoC FPGA, Booting of HPS and running Bare Metal Application from SD card“, 6. Sep. 2016, Supervisors: Dominique GIGI and Petr Zejdl.

[26] M. Ruiz und A. Carpeño, „Embedded Systems, Using Quartus and Buildroot for building Embedded Linux Systems (De1-SOC)“, 2017, V1.9.

[27] *Memory Initialization File (.mif)*, University of Florida,

[28] (). Embedded Linux Beginners Guide, Beginners guide to using embedded Linux on Altera SoC devices, RocketBoards.org, Adresse: https://rocketboards.org/foswiki/Documentation/EmbeddedLinuxBeginnerSGuide (besucht am 30. 08. 2018).

[29] (). Linux development repository for socfpga. socfpga-4.9.78-ltsi, Altera Opensource, Adresse: https://github.com/altera-opensource/linux-socfpga (besucht am 01. 09. 2018).

[30] *Using Linux on the DE1-SoC, For Quartus II 16.0*, Altera, Mai 2016.

[31] M. Gräfe, „C und Linux, Die Möglichkeiten des Betriebssystems mit eigenen Programmen nutzen“, in, 4. Aufl. Carl Hanser Verlag, 2010, ISBN: 978-3-446-42176-9.

A. Übersicht über elektronische Anhänge

- Bachelorarbeit in elektronischer Form
- Bitstreams
  - plasma_soc.rbf
- dts
  - socfpga_cyclone5_de1_soc.dtb
  - socfpga_cyclone5_de1_soc.dts
  - socfpga_cyclone5.dtsi
  - socfpga.dtsi
  - skeleton.dtsi
- Plasma-SoC - Quelltexte
  - proj - Vivado und Quartus Projektdateien
  - rtl - Hardwarebeschreibung
  - sw - Software
  - tb - Testbenches
  - dependencies
    - KebabOS - Firmware-Quellcode
    - PlasmaConvert - ELF-Converter

- Referenzen
- Webquellen
- README.md
- linux-socfpga-socfpga-4.9.78-ltsi.zip
  - Linux-Kernel-Quelltexte 4.9.78-ltsi
    - Als ZIP mitgeliefert, da auf dem MacOS das Dateisystem nicht Case-Sensitiv ist.
  - ubuntu-4.9.78.img.gz - modifiziertes Linux-Systemabbild
    - Komprimiertes Image, zum direkten Beschreiben eines Gerätes bitte `gunzip -c ubuntu-4.9.78.img.gz | pv | sudo dd of=/dev/sdf bs=1M` ausführen